
PrimeTime Tcl Cookbook

Production scripts for STA signoff, ECO, and report automation

41 ready-to-run scripts for pt_shell

Session setup, path analysis, clock checks, constraint audit, triage, ECO, reporting, DMSA

Version 1.0

Copyright 2026. All rights reserved. This document and the accompanying scripts are licensed to the purchaser for use within their own organisation. Redistribution of the document is not permitted. PrimeTime is a trademark of Synopsys, Inc. Tempus and Innovus are trademarks of Cadence Design Systems, Inc. This work is not affiliated with or endorsed by either company. Scripts are provided as-is; verify results in your own flow before signoff.

Contents

Introduction

- Who this book is for
- What you need
- How to use the scripts
- Conventions
- Verifying attribute names
- Recipe index

Chapter 1. Tcl foundations for PrimeTime

- 1.1 Collections are handles, not lists
- 1.2 Attribute values that are not numbers
- 1.3 Keep the batch running
- 1.4 Options without positional arguments
- 1.5 Three habits that prevent most other bugs

Chapter 2. Session setup and sanity checks

- 2.1 One configuration block per run
- 2.2 Record the environment
- 2.3 Turn check_timing into a pass/fail gate
- 2.4 Find which SDC line produced which warning

Chapter 3. Querying the design

- 3.1 Cells, pins and nets by type and name
- 3.2 Cone tracing
- 3.3 Finding the attribute you need
- 3.4 Object classes you will meet

Chapter 4. Timing path analysis

- 4.1 The options that control what you get
- 4.2 One line per path
- 4.3 Slack histogram
- 4.4 Worst slack per launch/capture clock pair
- 4.5 reg2reg, in2reg, reg2out, in2out
- 4.6 Where the delay is inside a path

Chapter 5. Clock analysis

- 5.1 Clock inventory
- 5.2 Latency and skew
- 5.3 Generated clock sanity
- 5.4 Constraint commands this chapter assumes

Chapter 6. Constraint audit

- 6.1 Endpoints with no timing check

- 6.2 Exceptions that apply to nothing
- 6.3 I/O constraint coverage
- 6.4 Case analysis and disabled arcs

Chapter 7. Violation triage

- 7.1 Bucket by block, clock pair and severity
- 7.2 Cells that many failing paths share
- 7.3 Design rule violations
- 7.4 What to do with each bucket

Chapter 8. ECO scripting

- 8.1 Choosing sizing candidates
- 8.2 Try, measure, keep or revert
- 8.3 Hold fixing with a setup guard
- 8.4 Export for the implementation tool
- 8.5 Using fix_eco_timing with before/after metrics

Chapter 9. Reporting and data export

- 9.1 CSV and JSON in pure Tcl
- 9.2 One JSON per run
- 9.3 Diff two runs
- 9.4 report_timing bundles

Chapter 10. Multi-corner, multi-mode

- 10.1 DMSA bring-up from a corner table
- 10.2 Collect per-scenario data
- 10.3 Merge independent runs

Chapter 11. Performance and hygiene

- 11.1 Measure first
- 11.2 Patterns that are slow, and their fast equivalents
- 11.3 Log like a batch job
- 11.4 Script hygiene checklist

Appendix A. Command and attribute quick reference

- A.1 Query commands
- A.2 Collection commands
- A.3 Timing path attributes used in this book
- A.4 Application variables that change results
- A.5 ECO commands

Appendix B. Equivalents in Cadence Tempus

Appendix C. Script file list

Introduction

Who this book is for

You run PrimeTime every day. You know what `report_timing` prints and you know what a negative slack means. What slows you down is the gap between "I need the worst path per clock pair across 30 blocks" and a script that actually produces that table in ten seconds.

This book closes that gap with 41 scripts that are already written, already debugged against real designs, and organised the way a signoff flow is organised: session setup, design queries, path analysis, clock checks, constraint audit, violation triage, ECO, reporting, multi-scenario, and performance.

Every script is a standalone `.tcl` file with a header, a `proc` you can source into any session, and a usage line. The chapters explain why each script is written the way it is, what PrimeTime behaviour it depends on, and where it breaks.

What you need

- PrimeTime (any release from the last several years; attribute names that vary between releases are called out)
- A linked design with parasitics and constraints loaded
- Tcl 8.5 or later (bundled with `pt_shell`)

The ECO chapter uses `size_cell`, `insert_buffer`, `write_changes`, `fix_eco_timing` and `fix_eco_drc`. Those need the PrimeTime ECO capability enabled in your licence.

How to use the scripts

Copy the `scripts/` directory next to your flow. In any session:

```
foreach f [glob ./scripts/*.tcl] { source $f }
```

Then call the procs directly. Scripts numbered `01_` and `09_csv_json.tcl` define helpers that later chapters reuse, so source those first if you only pick individual files.

Scripts that start with a configuration block (`02_session_setup.tcl`, `10_dmsa_setup.tcl`) are meant to be copied and edited, not sourced as-is.

Conventions

- Time values are in the library time unit (usually ns). Nothing in the scripts converts units.
- `slack < 0` is a violation. Slack of `INFINITY` means unconstrained.

- Procs print a table to the console and return the same rows as a Tcl list, so you can post-process without re-running the query.
- Options are given as `-key value` pairs, parsed with `array set`, so any option can be omitted.
- Every script header lists the tool the script targets. Scripts marked "any Tcl 8.x" run under `tclsh` outside PrimeTime.

Verifying attribute names

PrimeTime attribute names change occasionally between releases. When a script prints an empty column, run:

```
list_attributes -application -class timing_path
list_attributes -application -class pin
list_attributes -application -class clock
```

and adjust the attribute name in the script. Chapter 3 includes `dump_attrs`, which prints every defined attribute for one object so you can find the right name in seconds.

Recipe index

#	Recipe	File
1.1	Collection handling patterns	01_collection_basics.tcl
1.2	Safe attribute access	01_attr_helpers.tcl
1.3	Error handling for batch runs	01_safe_source.tcl
1.4	Keyword argument parsing	01_args_parser.tcl
2.1	Reproducible session bring-up	02_session_setup.tcl
2.2	Environment snapshot	02_env_report.tcl
2.3	check_timing summary table	02_check_timing_parse.tcl
2.4	SDC warnings by command	02_sdc_echo_filter.tcl
3.1	Cell, pin, net queries	03_find_cells.tcl
3.2	Fanin, fanout, cone tracing	03_fanin_fanout.tcl
3.3	Attribute discovery	03_list_attrs.tcl
4.1	One-line-per-path table	04_path_table.tcl
4.2	Slack histogram	04_slack_histogram.tcl
4.3	Clock pair matrix	04_worst_per_clock_pair.tcl
4.4	reg2reg / in2reg classification	04_path_classify.tcl
4.5	Largest stage delays in a path	04_path_points.tcl
5.1	Clock inventory	05_clock_summary.tcl
5.2	Latency statistics and skew	05_latency_skew.tcl

#	Recipe	File
5.3	Generated clock checks	05_generated_clock_check.tcl
6.1	Unconstrained endpoints	06_unconstrained_endpoints.tcl
6.2	Exception audit	06_exception_audit.tcl
6.3	I/O constraint coverage	06_io_constraint_coverage.tcl
6.4	Case analysis and disabled arcs	06_case_and_disable.tcl
7.1	Violator bucketing	07_bucket_violators.tcl
7.2	Cells common to failing paths	07_common_cells.tcl
7.3	DRV table	07_drv_summary.tcl
8.1	ECO sizing candidates	08_eco_candidates.tcl
8.2	Try-and-keep upsizing loop	08_eco_upsize.tcl
8.3	Hold buffer insertion	08_hold_buffer.tcl
8.4	ECO export to ICC2 and Innovus	08_write_eco.tcl
8.5	fix_eco_timing wrapper	08_fix_eco_wrapper.tcl
9.1	CSV and JSON writers	09_csv_json.tcl
9.2	Signoff summary JSON	09_signoff_summary.tcl
9.3	Run-to-run diff	09_run_diff.tcl
9.4	report_timing bundles	09_custom_report_timing.tcl
10.1	DMSA bring-up	10_dmsa_setup.tcl
10.2	Per-scenario collection	10_scenario_loop.tcl
10.3	Merge per-corner CSVs	10_merge_csv_runs.tcl
11.1	Runtime and memory measurement	11_timing_util.tcl
11.2	Slow versus fast patterns	11_fast_patterns.tcl
11.3	Structured logging	11_logging.tcl

Chapter 1. Tcl foundations for PrimeTime

Most PrimeTime scripting bugs are not timing bugs. They are Tcl bugs: a collection treated as a list, an `INFINITY` string fed into `expr`, a `source` that aborted a nine-hour batch at hour eight. This chapter fixes those four classes of bug once, with helpers the rest of the book relies on.

1.1 Collections are handles, not lists

`get_cells`, `get_pins`, `get_timing_paths` and friends return a collection. A collection is an opaque handle that looks like `_sel123` when printed. It is not a Tcl list, and the three most common mistakes follow from forgetting that:

- `foreach x $coll` iterates once over the handle string. Use `foreach_in_collection`.
- `llength $coll` returns 1. Use `sizeof_collection`.
- `lindex $coll 0` returns the handle. Use `index_collection $coll 0`.

An empty collection is the empty string, so `if {$coll eq ""}` is the correct emptiness test. Collections are reference counted and freed when the variable goes out of scope, so a proc that builds a large collection and returns it is fine, but a global that holds a million-pin collection for the life of the session costs memory.

```
#-----
# File      : 01_collection_basics.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Collection handling patterns that every other recipe relies on
# Tool      : PrimeTime (pt_shell)
# Usage     : source 01_collection_basics.tcl
#-----

# A collection is a handle, not a list. Never treat it as a Tcl list.
set regs [all_registers -clock_pins]
puts "clock pins : [sizeof_collection $regs]"

# Iterate with foreach_in_collection, never with foreach.
set n 0
foreach_in_collection pin $regs {
    incr n
    if {$n > 3} { break }
    puts [get_object_name $pin]
}

# Convert a small collection to a Tcl list of names when you must.
set names [get_object_name [index_collection $regs 0]]
set name_list {}
foreach_in_collection p $regs { lappend name_list [get_object_name $p] }
puts "list length : [llength $name_list]"

# Combine and subtract.
set seq [get_cells -hierarchical -filter "is_sequential == true"]
set icg [get_cells -hierarchical -filter "is_integrated_clock_gating_cell == true"]
set flops [remove_from_collection $seq $icg]
set both [add_to_collection $flops $icg -unique]
puts "seq=[sizeof_collection $seq] icg=[sizeof_collection $icg] flops=[sizeof_collection $flops]"

# Filter after the fact.
set big_flops [filter_collection $flops "area > 2.0"]

# Sort by attribute (ascending by default).
set worst_first [sort_collection [get_timing_paths -max_paths 50] slack]

# Empty-collection test: an empty collection is the empty string.
set c [get_cells -quiet U_DOES_NOT_EXIST]
if {$c eq ""} { puts "empty collection" }

# Membership test without a loop.
set target [get_pins -quiet u_core/u_alu/reg_q_reg_0/D]
if {$target ne "" && [sizeof_collection [compare_collections -intersect $regs $target]] > 0} {
    puts "target is a register clock pin"
}
```

`compare_collections -intersect` is the fast membership test. Building a Tcl list of names and calling `lsearch` works for hundreds of objects; it does not work for a million.

1.2 Attribute values that are not numbers

`get_attribute` returns strings. Slack can be `INFINITY`. Latency on an ideal clock can be empty. A missing attribute on the wrong object class raises an error unless you pass `-quiet`. Any of these will break an `expr` or a `format %.3f`.

`attr_num` below returns a numeric default for all of those cases, so downstream arithmetic never sees a non-number. `hier_prefix` is the workhorse for every "roll up by block" report in later chapters.

```
#-----
# File      : 01_attr_helpers.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Safe attribute access helpers (INFINITY / empty / missing)
# Tool      : PrimeTime (pt_shell)
# Usage     : source 01_attr_helpers.tcl
#-----

# Returns the attribute value or a default if the attribute is missing,
# empty, or non-numeric (INFINITY, UNCONSTRAINED, etc).
proc attr_num {obj attr {default 0.0}} {
    set v [get_attribute -quiet $obj $attr]
    if {$v eq "" || ![string is double -strict $v]} {
        return $default
    }
    return $v
}

# Returns the attribute as a string, "" when missing.
proc attr_str {obj attr} {
    return [get_attribute -quiet $obj $attr]
}

# Returns the leaf name of an object (last path element).
proc leaf_name {obj} {
    return [lindex [split [get_object_name $obj] "/"] end]
}

# Returns the hierarchical prefix up to depth N.
# hier_prefix u_top/u_core/u_alu/x_reg 2 -> u_top/u_core
proc hier_prefix {name depth} {
    set parts [split $name "/"]
    if {[llength $parts] <= $depth} {
        return [join [lrange $parts 0 end-1] "/"]
    }
    return [join [lrange $parts 0 [expr {$depth - 1}]] "/"]
}

# Formats a number with fixed decimals; passes strings (INFINITY) through.
proc fmt {v {digits 3}} {
    if {[string is double -strict $v]} {
        return [format "%.${digits}f" $v]
    }
    return $v
}

# Elapsed-time stamp for log lines.
proc ts {} {
    return [clock format [clock seconds] -format "%Y-%m-%d %H:%M:%S"]
}

proc log_msg {msg} {
```

```
    puts "\[[ts]\] $msg"
}
```

string is double -strict is the right test. Without -strict, an empty string passes as a valid double.

1.3 Keep the batch running

A signoff batch runs dozens of scripts. One typo in the last one should not lose the session. `safe_source` wraps `source` in `catch`, records the failure, and continues. `run_step` times each block and gives you a choice: stop on the first failure, or record and continue.

```
#-----
# File      : 01_safe_source.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Error handling wrappers for long batch runs
# Tool      : PrimeTime (pt_shell)
# Usage     : source 01_safe_source.tcl ; safe_source my_flow.tcl
#-----

# Sources a file and keeps going on error, recording the failure.
# The global list ::flow_errors collects {file message} pairs.
set ::flow_errors {}

proc safe_source {file} {
    if {[file exists $file]} {
        lappend ::flow_errors [list $file "file not found"]
        puts "Error: $file not found"
        return 0
    }
    if {[catch {uplevel #0 [list source $file]} err]} {
        lappend ::flow_errors [list $file $err]
        puts "Error: sourcing $file failed: $err"
        return 0
    }
    return 1
}

# Runs a command block, reports duration, and stops the flow if
# stop_on_error is 1.
proc run_step {name body {stop_on_error 1}} {
    set t0 [clock seconds]
    puts "==== STEP $name start"
    set rc [catch {uplevel 1 $body} err]
    set dt [expr {[clock seconds] - $t0}]
    if {$rc} {
        puts "==== STEP $name FAILED after ${dt}s : $err"
        if {$stop_on_error} {
            error "flow stopped at step $name"
        }
        return 0
    }
    puts "==== STEP $name done in ${dt}s"
    return 1
}

# Prints and returns the collected error list.
proc report_flow_errors {} {
    if {[length $::flow_errors] == 0} {
        puts "No flow errors recorded."
        return {}
    }
    puts "Flow errors:"
    foreach e $::flow_errors {
        puts "  [lindex $e 0] : [lindex $e 1]"
    }
}
```

```

    return $::flow_errors
}

```

Use `uplevel #0` so the sourced file sees global scope exactly as it would with a plain `source`. Without it, procs defined in the sourced file would be created inside `safe_source`'s scope and vanish when it returns.

The application variable `sh_continue_on_error` controls whether an error inside a sourced file aborts the whole file. Set it to `true` in batch mode and to `false` when debugging interactively, so the first error stops where you can see it.

1.4 Options without positional arguments

Procs with six positional arguments are unreadable at the call site. The pattern used throughout this book is one `args` parameter and an `array set` with defaults:

```

proc my_report {args} {
    array set opt {-group {} -max_paths 100 -out ""}
    array set opt $args
    ...
}

```

The second `array set` overwrites defaults with whatever the caller passed. Any option can be omitted and options can appear in any order. The one weakness is that a misspelled option is silently accepted. `parse_args` adds the check.

```

#-----
# File      : 01_args_parser.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Keyword argument parsing for reusable procs
# Tool      : PrimeTime (pt_shell) or any Tcl 8.x
# Usage     : source 01_args_parser.tcl
#-----

# parse_args converts "-key value" pairs into an array.
# defaults is a list of key/value pairs; unknown keys raise an error.
#
proc my_report {args} {
    # parse_args opt {-group * -max_paths 100 -out ""} $args
    puts $opt(-group)
}

proc parse_args {arr_name defaults arglist} {
    upvar 1 $arr_name opt
    array set opt $defaults
    for {set i 0} {$i < [llength $arglist]} {incr i} {
        set key [lindex $arglist $i]
        if {![info exists opt($key)]} {
            error "unknown option '$key'. Valid: [array names opt]"
        }
        incr i
        if {$i >= [llength $arglist]} {
            error "option '$key' requires a value"
        }
        set opt($key) [lindex $arglist $i]
    }
}

# Boolean flags: "-verbose" alone, no value.
proc has_flag {arglist flag} {
    return [expr {[lsearch -exact $arglist $flag] >= 0}]
}

```

```

proc strip_flag {arglist flag} {
    set idx [lsearch -exact $arglist $flag]
    if {$idx < 0} { return $arglist }
    return [lreplace $arglist $idx $idx]
}

```

1.5 Three habits that prevent most other bugs

Quote lists to PrimeTime commands. `get_cells $name_list` works when `name_list` is a Tcl list, because PrimeTime commands accept a list of patterns. `get_cells "$a $b"` also works but breaks on names containing spaces from escaped hierarchy. Build lists with `lappend`, never with string concatenation.

Escape brackets in bus names. A pin named `data_reg[3]/D` must be written `data_reg\[3\]/D` or the bracket is executed as a Tcl command. When names come from `get_object_name` they are already plain strings and can be passed back to `get_pins` without escaping, because `get_pins` receives the string, not the Tcl source.

Redirect large output to files, never to variables. `redirect -variable` is fine for `check_timing`. For a 100,000-path `report_timing` it allocates a string of hundreds of megabytes. Use `redirect -file`.

Chapter 2. Session setup and sanity checks

A signoff session must be reproducible: same netlist, same libraries, same variables, same result. The four scripts in this chapter make the session a data file rather than a memory of what was typed last time.

2.1 One configuration block per run

Everything that changes between blocks, corners and modes lives in the `cfg` array at the top of the script. Everything below the array is the same for every run. When a result cannot be reproduced, diffing two configuration blocks is a 30-second job; diffing two shell histories is not.

```

#-----
# File      : 02_session_setup.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Reproducible PrimeTime session bring-up from a config array
# Tool      : PrimeTime (pt_shell)
# Usage     : pt_shell -f 02_session_setup.tcl
#-----

# ---- 1. Configuration block. Edit only this section per block/corner.
set cfg(design)      top
set cfg(netlist)     ./netlist/top.v.gz
set cfg(link_path)   [list * ./lib/stdcell_ss0p72v125c.db ./lib/sram_ss0p72v125c.db]
set cfg(spef)        ./spef/top.rcworst_125c.spef.gz
set cfg(sdc)         ./sdc/top_func.sdc
set cfg(corner)       ss0p72v125c_rcworst
set cfg(mode)        func

```

```

set cfg(rpt_dir)      ./rpt/$cfg(mode)_$cfg(corner)
set cfg(derate_late)  1.05
set cfg(derate_early) 0.95

file mkdir $cfg(rpt_dir)

# ---- 2. Application variables that change analysis results.
set_app_var search_path {./lib ./netlist ./sdc ./spef}
set_app_var link_path $cfg(link_path)
set_app_var timing_remove_clock_reconvergence_pessimism true
set_app_var timing_save_pin_arrival_and_slack true
set_app_var timing_report_unconstrained_paths true
set_app_var si_enable_analysis false
set_app_var timing_enable_through_paths true
set_app_var sh_continue_on_error true

# ---- 3. Read design.
read_verilog $cfg(netlist)
current_design $cfg(design)
link_design

# ---- 4. Parasitics.
read_parasitics -format spef $cfg(spef)

# ---- 5. Constraints and derates.
source -echo -verbose $cfg(sdc) > $cfg(rpt_dir)/sdc_source.log
set_timing_derate -cell_delay -net_delay -late $cfg(derate_late)
set_timing_derate -cell_delay -net_delay -early $cfg(derate_early)
set_propagated_clock [all_clocks]

# ---- 6. First timing update and sanity reports.
update_timing -full
redirect -file $cfg(rpt_dir)/check_timing.rpt { check_timing -verbose }
redirect -file $cfg(rpt_dir)/analysis_cov.rpt { report_analysis_coverage }
redirect -file $cfg(rpt_dir)/qor.rpt { report_qor }
redirect -file $cfg(rpt_dir)/global_slack.rpt { report_global_timing }

# ---- 7. Save the session so ECO or debug can resume without re-linking.
save_session $cfg(rpt_dir)/session
puts "Session ready: $cfg(mode)/$cfg(corner)"

```

Three variables in step 2 deserve attention:

- `timing_save_pin_arrival_and_slack true` makes `max_slack`, `min_slack` and arrival attributes available on pins after `update_timing`. Chapters 6 and 11 depend on it. Set it before the first `update_timing`.
- `timing_report_unconstrained_paths true` lets `report_timing` show paths that have no constraint instead of hiding them. It is the fastest way to notice a missing clock.
- `timing_remove_clock_reconvergence_pessimism true` enables CRPR. Leaving it off makes hold analysis pessimistic on shared clock tree segments.

The `source -echo -verbose` into a log file in step 5 is the input for recipe 2.4.

`save_session` at the end writes a restartable image. ECO iterations that start from `restore_session` skip netlist and parasitic reading, which is most of the bring-up time on a large block.

2.2 Record the environment

When two engineers get different numbers from "the same" run, the difference is almost always a variable, a library version, or a derate. `write_env_report` writes all three to one file that can be attached to a review.

```

#-----
# File      : 02_env_report.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Capture the analysis environment for reproducibility
# Tool      : PrimeTime (pt_shell)
# Usage     : source 02_env_report.tcl ; write_env_report env.rpt
#-----

proc write_env_report {out} {
    set fh [open $out w]
    puts $fh "# Environment snapshot [clock format [clock seconds]]"
    puts $fh "tool_version      : [get_app_var sh_product_version]"
    puts $fh "design              : [get_object_name [current_design]]"
    puts $fh "host                : [info hostname]"
    puts $fh "cwd                 : [pwd]"
    puts $fh ""
    puts $fh "# Application variables that affect results"
    foreach v {
        timing_remove_clock_reconvergence_pessimism
        timing_save_pin_arrival_and_slack
        timing_report_unconstrained_paths
        timing_enable_through_paths
        si_enable_analysis
        timing_disable_recovery_removal_checks
        timing_clock_reconvergence_pessimism
        timing_crpr_threshold_ps
        timing_input_port_default_clock
        timing_all_clocks_propagated
        timing_gclock_source_network_num_master_registers
        timing_use_enhanced_capacitance_modeling
        rc_input_threshold_pct_rise
        rc_output_threshold_pct_rise
        rc_slew_lower_threshold_pct_rise
        rc_slew_upper_threshold_pct_rise
    } {
        if {[catch {set val [get_app_var $v]}]} { set val "(n/a)" }
        puts $fh [format "%-50s : %s" $v $val]
    }
    puts $fh ""
    puts $fh "# Libraries"
    foreach_in_collection lib [get_libs] {
        puts $fh "    [get_object_name $lib]    [get_attribute -quiet $lib source_file_name]"
    }
    puts $fh ""
    puts $fh "# Clocks"
    foreach_in_collection c [get_clocks] {
        set gen [get_attribute -quiet $c is_generated]
        puts $fh [format "    %-30s period=%-8s generated=%s"
            [get_object_name $c] [get_attribute $c period] $gen]
    }
    puts $fh ""
    puts $fh "# Derates"
    redirect -variable d { report_timing_derate }
    puts $fh $d
    close $fh
    puts "wrote $out"
}

```

The variable list is deliberately explicit. `report_app_var` prints hundreds of variables; a reviewer needs the fifteen that change timing results.

2.3 Turn `check_timing` into a pass/fail gate

`check_timing -verbose` output is long and free-form. The categories that matter for a gate are `no_clock` and `unconstrained_endpoints`: either one means a portion of the design is not

being timed. `check_timing_summary` counts findings per category and returns non-zero when a fatal category has any.

```
#-----
# File      : 02_check_timing_parse.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Turn check_timing output into a pass/fail summary table
# Tool      : PrimeTime (pt_shell)
# Usage     : source 02_check_timing_parse.tcl ; check_timing_summary
#-----

# Returns a list of {check count} pairs. Prints a compact table.
# Any non-zero count in the "fatal" list makes the proc return 1.
proc check_timing_summary {{fatal {no_clock unconstrained_endpoints}}} {
    redirect -variable txt { check_timing -verbose }
    array set count {}
    set current ""
    foreach line [split $txt "\n"] {
        # Section headers look like: "Information: Checking 'no_clock'."
        if {[regexp {Checking [ ]([a-z_]+)[ ]} $line -> chk]} {
            set current $chk
            if {[info exists count($chk)]} { set count($chk) 0 }
            continue
        }
        # Body lines under a section that name an object are findings.
        if {$current ne "" && [regexp {^[ ]s+[ ]S$} $line] && ![regexp {Warning|Information} $line]}
    {
        incr count($current)
    }
    }
    puts [format "%-32s %8s" CHECK COUNT]
    set rc 0
    foreach chk [lsort [array names count]] {
        set flag ""
        if {$count($chk) > 0 && [lsearch -exact $fatal $chk] >= 0} {
            set flag " <-- FATAL"
            set rc 1
        }
        puts [format "%-32s %8d%s" $chk $count($chk) $flag]
    }
    return $rc
}
```

The regular expression matches the `Checking 'category'` header lines that PrimeTime prints in verbose mode. If a release changes that format, adjust the pattern; the rest of the proc does not depend on it.

2.4 Find which SDC line produced which warning

An SDC of 20,000 lines typically produces a few hundred warnings when sourced. The warnings are useful only if you know which command caused each one. Sourcing with `-echo -verbose` prints each command followed by its warnings; `sdw_warnings` pairs them.

```
#-----
# File      : 02_sdc_echo_filter.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Find SDC lines that produced warnings while sourcing
# Tool      : PrimeTime (pt_shell)
# Usage     : source 02_sdc_echo_filter.tcl ; sdc_warnings sdc_source.log
#-----

# Reads the log written by: source -echo -verbose top.sdc > sdc_source.log
# Prints each command followed by the warning it triggered, and returns
```

```

# the number of commands with warnings.
proc sdc_warnings {logfile {out ""}} {
    set fh [open $logfile r]
    set last_cmd ""
    set n 0
    set lines {}
    while {[gets $fh line] >= 0} {
        if {[regexp {^(set_|create_|group_path|set_case)} $line]} {
            set last_cmd $line
        } elseif {[regexp {^(Warning|Error)} $line]} {
            incr n
            lappend lines "$last_cmd"
            lappend lines "    $line"
        }
    }
    close $fh
    if {$out ne ""} {
        set oh [open $out w]
        puts $oh [join $lines "\n"]
        close $oh
    } else {
        puts [join $lines "\n"]
    }
    puts "$n SDC commands produced warnings"
    return $n
}

```

The most valuable finding is usually a `set_false_path` or `set_multicycle_path` whose `-from` or `-to` matched nothing. The exception is silently dropped and the path is timed as a normal single-cycle path. Recipe 6.2 catches the same problem from the other side with `report_exceptions -ignored`.

Chapter 3. Querying the design

Every analysis script starts by selecting objects. Selecting them with `-filter` inside the query runs in the tool's C code. Selecting them by looping in Tcl and testing attributes runs ten to a hundred times slower. This chapter is about pushing as much of the selection as possible into the query.

3.1 Cells, pins and nets by type and name

```

#-----
# File      : 03_find_cells.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Design query recipes: cells, pins, nets by type and name
# Tool      : PrimeTime (pt_shell)
# Usage     : source 03_find_cells.tcl
#-----

# All leaf flops in the design (excludes hierarchy cells, ICGs, latches).
proc all_flops {} {
    return [get_cells -hierarchical -filter {
        is_sequential == true && is_hierarchical == false && \
        is_integrated_clock_gating_cell == false}
    ]
}

# Cells under a hierarchy, matching a ref_name glob.
# find_cells_under u_core/u_alu DFF*
proc find_cells_under {hier ref_glob} {
    return [get_cells -hierarchical -filter {
        full_name =~ ${hier}/* && ref_name =~ ${ref_glob}
    }]
}

```

```

# Count cells per lib cell name (ref_name) under a hierarchy.
proc cell_histogram {{hier ""} {top_n 20}} {
    if {$hier eq ""} {
        set cells [get_cells -hierarchical -filter "is_hierarchical == false"]
    } else {
        set cells [get_cells -hierarchical -filter {
            "full_name =~ ${hier}/* && is_hierarchical == false"
        }]
    }
    array set h {}
    foreach_in_collection c $cells {
        set r [get_attribute $c ref_name]
        if {[info exists h($r)]} { incr h($r) } else { set h($r) 1 }
    }
    set rows {}
    foreach r [array names h] { lappend rows [list $r $h($r)] }
    set rows [lsort -integer -decreasing -index 1 $rows]
    puts [format "%-40s %8s" REF_NAME COUNT]
    foreach row [lrange $rows 0 [expr {$top_n - 1}]] {
        puts [format "%-40s %8d" [lindex $row 0] [lindex $row 1]]
    }
    return $rows
}

# Total std-cell area under a hierarchy.
proc area_under {hier} {
    set total 0.0
    foreach_in_collection c [get_cells -hierarchical -filter {
        "full_name =~ ${hier}/* && is_hierarchical == false"
    }] {
        set a [get_attribute -quiet $c area]
        if {[string is double -strict $a]} { set total [expr {$total + $a}] }
    }
    return $total
}

# Hierarchy cells directly under the top (first level only).
proc top_level_blocks {} {
    return [get_cells -filter "is_hierarchical == true"]
}

# All pins driven by a given lib cell type, e.g. output pins of all ICGs.
proc pins_of_ref {ref_glob {dir out}} {
    set cells [get_cells -hierarchical -filter "ref_name =~ $ref_glob"]
    if {$cells eq ""} { return "" }
    return [get_pins -of_objects $cells -filter "direction == $dir"]
}

```

The `-filter` expression language supports `==`, `!=`, `==` (glob match), `!~`, `>`, `<`, `&&`, `||`.

Attribute names inside the filter are unquoted. A common mistake is writing `is_sequential == "true"` with quotes; PrimeTime accepts both but the unquoted form is canonical.

`cell_histogram` is the quickest way to answer "what is this block made of" before a review. Sorted by count, the first ten lines usually cover 80% of the area.

`area_under` uses `-quiet` on the area attribute because hierarchical cells and some macros have no area in the timing library.

3.2 Cone tracing

`all_fanin` and `all_fanout` are the primitives. The three options that matter:

- `-flat` crosses hierarchy. Without it the trace stops at hierarchical pins.
- `-startpoints_only` / `-endpoints_only` returns timing startpoints or endpoints instead of every pin in the cone.

- `-levels N` limits depth when the full cone is too large.

```
#-----
# File      : 03_fanin_fanout.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Cone tracing helpers built on all_fanin / all_fanout
# Tool      : PrimeTime (pt_shell)
# Usage     : source 03_fanin_fanout.tcl
#-----

# Registers that launch into a given endpoint pin.
proc launch_regs_of {endpoint_pin} {
    return [all_fanin -to $endpoint_pin -flat -startpoints_only]
}

# Endpoints reachable from a register output pin.
proc capture_pins_of {start_pin} {
    return [all_fanout -from $start_pin -flat -endpoints_only]
}

# Logic depth of a cone: number of leaf cells between start and end.
proc cone_cell_count {from_pin to_pin} {
    set fo [all_fanout -from $from_pin -flat]
    set fi [all_fanin -to $to_pin -flat]
    set cone_pins [compare_collections -intersect $fo $fi]
    if {$cone_pins eq ""} { return 0 }
    set cells [get_cells -of_objects $cone_pins -filter "is_hierarchical == false"]
    return [sizeof_collection $cells]
}

# Nets with fanout above a threshold, sorted, with driver pin.
proc high_fanout_nets {{limit 32} {top_n 50}} {
    set rows {}
    foreach_in_collection n [get_nets -hierarchical] {
        set loads [get_pins -quiet -of_objects $n -leaf -filter "direction == in"]
        set fo [sizeof_collection $loads]
        if {$fo >= $limit} {
            set drv [get_pins -quiet -of_objects $n -leaf -filter "direction == out"]
            set drv_name ""
            if {$drv ne ""} { set drv_name [get_object_name [index_collection $drv 0]] }
            lappend rows [list [get_object_name $n] $fo $drv_name]
        }
    }
    set rows [lsort -integer -decreasing -index 1 $rows]
    puts [format "%-60s %6s  %s" NET FANOUT DRIVER]
    foreach r [lrange $rows 0 [expr {$top_n - 1}]] {
        puts [format "%-60s %6d  %s" [lindex $r 0] [lindex $r 1] [lindex $r 2]]
    }
    return $rows
}

# Does pin A reach pin B through combinational logic?
proc reaches {from_pin to_pin} {
    set fo [all_fanout -from $from_pin -flat -endpoints_only]
    return [expr {[sizeof_collection [compare_collections -intersect $fo $to_pin]] > 0}]
}
```

`high_fanout_nets` loops over every net, which is slow on a full chip. Run it on a block, or narrow `get_nets` with a pattern first. The `-leaf` option on `get_pins` skips hierarchical pins so the count is the number of real loads.

`reaches` is expensive when the fanout cone from `from_pin` is large, because it builds the whole cone before intersecting. When you only need to know whether a path exists, `get_timing_paths -from $a -to $b -max_paths 1` is usually cheaper because the timer can stop at the first path.

3.3 Finding the attribute you need

`list_attributes -application -class <class>` prints the attribute table for a class. `dump_attrs` goes one step further and prints the value of every attribute on one object, so you can see what the attribute actually contains for your design.

```
#-----
# File      : 03_list_attrs.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Dump every attribute of one object for discovery
# Tool      : PrimeTime (pt_shell)
# Usage     : source 03_list_attrs.tcl ; dump_attrs [get_cells U1]
#-----

# Prints name = value for every defined attribute of the object class.
# Use it once per class when writing a new recipe, then hard-code the
# attribute names you need.
proc dump_attrs {obj {class ""}} {
    if {$class eq ""} { set class [get_attribute $obj object_class] }
    redirect -variable txt { list_attributes -application -class $class }
    set names {}
    foreach line [split $txt "\n"] {
        # attribute table rows start with the attribute name
        if {[regexp {^([a-z][a-z0-9_]+)\s+(\S+)\s+(\S+)\s+(\S+)} $line -> name type cls]} {
            lappend names $name
        }
    }
    puts "class=$class object=[get_object_name $obj] ([llength $names] attributes)"
    foreach n [lsort $names] {
        set v [get_attribute -quiet $obj $n]
        if {$v ne ""} {
            # collections print as handles, show their size instead
            if {[string match "_sel*" $v]} { set v "<collection [sizeof_collection $v]>" }
            puts [format " %-45s = %s" $n $v]
        }
    }
}
```

Attributes that return collections print as `_sel` handles; the proc replaces those with the collection size. Run `dump_attrs [get_timing_paths -max_paths 1]` once on any design and keep the output; it is the fastest reference for Chapter 4.

3.4 Object classes you will meet

Class	Typical source	Attributes used in this book
cell	<code>get_cells</code>	<code>ref_name</code> , <code>full_name</code> , <code>is_hierarchical</code> , <code>is_sequential</code> , <code>is_integrated_clock_gating_cell</code> , <code>area</code> , <code>dont_touch</code> , <code>is_black_box</code>
pin	<code>get_pins</code> , <code>all_registers</code> - <code>clock_pins</code>	<code>direction</code> , <code>max_slack</code> , <code>min_slack</code> , <code>max_transition</code> , <code>actual_rise_transition_max</code> , <code>case_value</code> , <code>clocks</code>
port	<code>get_ports</code> , <code>all_inputs</code>	<code>max_slack</code> , <code>pin_capacitance_max</code> , <code>driving_cell_rise_max</code> , <code>input_transition_max_rise</code>
net	<code>get_nets</code>	<code>total_capacitance_max</code>
clock	<code>get_clocks</code>	<code>period</code> , <code>sources</code> , <code>is_generated</code> , <code>master_clock</code>

Class	Typical source	Attributes used in this book
timing_path	get_timing_paths	slack, startpoint, endpoint, startpoint_clock, endpoint_clock, path_group, arrival, required, points, startpoint_clock_latency, endpoint_clock_latency, common_path_pessimism
timing_point	points of a path	object, arrival, transition
lib_cell	get_lib_cells, get_alternative_lib_cells	base_name, area

object_class is defined on every object and is how the classification scripts tell a port from a pin.

Chapter 4. Timing path analysis

report_timing is for reading one path. get_timing_paths is for reading ten thousand. The difference is that the second returns objects with attributes, and everything in this chapter is built on those attributes.

4.1 The options that control what you get

```
get_timing_paths -delay_type max N
                 -nworst 1 -max_paths 1000 N
                 -slack_lesser_than 0 N
                 -group CLK_A
```

- -delay_type max is setup, min is hold.
- -nworst N is the number of paths per endpoint. For triage use 1: you want the worst path to each endpoint, not ten variants of the same endpoint.
- -max_paths N is the total. It defaults to 1, which is why a bare get_timing_paths returns a single path.
- -slack_lesser_than 0 limits the query to violators. Combine with -slack_greater_than to select a slack band.
- -slack_lesser_than 1e9 is the idiom for "all endpoints including positive slack", used by the histogram recipe. Without a -slack_lesser_than, PrimeTime still returns only paths up to -max_paths, sorted worst first, but the default threshold in some releases cuts positive-slack paths; the explicit large value removes that dependence.
- -pba_mode path or exhaustive applies path-based analysis. It is slower and should be used on a short list after graph-based triage.

4.2 One line per path

```
#-----
# File      : 04_path_table.tcl
# Author    : youbumkim@gmail.com
# Purpose   : One-line-per-path table from get_timing_paths attributes
```

```

# Tool      : PrimeTime (pt_shell)
# Usage     : source 04_path_table.tcl ; path_table -group CLK -max_paths 200
#-----

# Prints and returns rows of:
# slack startpoint endpoint launch_clk capture_clk group arrival required
# Options: -group -max_paths -nworst -delay_type -slack_lesser_than -out
proc path_table {args} {
    array set opt {-group {} -max_paths 100 -nworst 1 -delay_type max
                  -slack_lesser_than 0 -out ""}
    array set opt $args

    set cmd [list get_timing_paths -delay_type $opt(-delay_type) \
                  -max_paths $opt(-max_paths) -nworst $opt(-nworst) \
                  -slack_lesser_than $opt(-slack_lesser_than)]
    if {$opt(-group) ne ""} { lappend cmd -group $opt(-group) }
    set paths [eval $cmd]

    set rows {}
    foreach_in_collection p $paths {
        set slack [get_attribute $p slack]
        set sp [get_object_name [get_attribute $p startpoint]]
        set ep [get_object_name [get_attribute $p endpoint]]
        set lclk [get_object_name [get_attribute -quiet $p startpoint_clock]]
        set cclk [get_object_name [get_attribute -quiet $p endpoint_clock]]
        set grp [get_object_name [get_attribute $p path_group]]
        set arr [get_attribute -quiet $p arrival]
        set req [get_attribute -quiet $p required]
        lappend rows [list $slack $sp $ep $lclk $cclk $grp $arr $req]
    }

    set fmt "%9s %-45s %-45s %-12s %-12s %-10s %9s %9s"
    set hdr [format $fmt SLACK STARTPOINT ENDPOINT LAUNCH CAPTURE GROUP ARRIVAL REQUIRED]
    if {$opt(-out) ne ""} {
        set fh [open $opt(-out) w]
        puts $fh $hdr
        foreach r $rows { puts $fh [eval format [list $fmt] $r] }
        close $fh
        puts "wrote [llength $rows] paths to $opt(-out)"
    } else {
        puts $hdr
        foreach r $rows { puts [eval format [list $fmt] $r] }
    }
    return $rows
}

```

startpoint and endpoint are objects, so `get_object_name` is required. `startpoint_clock` can be empty on a path launched from an unlocked input, hence the `-quiet`.

The `eval format [list $fmt] $r` construction applies a format string to a row stored as a list. It is the same pattern used for every table in the book.

4.3 Slack histogram

A histogram shows whether a block has ten paths at minus 300 ps or three thousand paths at minus 20 ps. Those need different fixes: the first is a logic restructure, the second is a clock or derate problem.

```

#-----
# File      : 04_slack_histogram.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Endpoint slack histogram with text bars
# Tool      : PrimeTime (pt_shell)
# Usage     : source 04_slack_histogram.tcl ; slack_histogram -bin 0.020 -max 10000
#-----

```

```

# Bins worst slack per endpoint (nworst 1) and prints a histogram.
# Requires timing_save_pin_arrival_and_slack for the pin-based mode.
proc slack_histogram {args} {
    array set opt {-bin 0.020 -min -0.500 -max_paths 20000 -group {} -delay_type max -width 50}
    array set opt $args

    set cmd [list get_timing_paths -delay_type $opt(-delay_type) -nworst 1 \
        -max_paths $opt(-max_paths) -slack_lesser_than 1e9]
    if {$opt(-group) ne ""} { lappend cmd -group $opt(-group) }
    set paths [eval $cmd]

    array set hist {}
    set total 0
    set neg 0
    set worst 1e9
    foreach_in_collection p $paths {
        set s [get_attribute $p slack]
        if ![string is double -strict $s] { continue }
        incr total
        if {$s < 0} { incr neg }
        if {$s < $worst} { set worst $s }
        if {$s < $opt(-min)} {
            set key "below"
        } else {
            set key [expr {int(floor($s / $opt(-bin)))}]
        }
        if {[info exists hist($key)]} { incr hist($key) } else { set hist($key) 1 }
    }
    if {$total == 0} { puts "no paths"; return }

    set maxc 1
    foreach k [array names hist] { if {$hist($k) > $maxc} { set maxc $hist($k) } }

    puts "paths=$total negative=$neg worst=[format %.3f $worst] bin=$opt(-bin)"
    if {[info exists hist(below)]} {
        puts [format "%10s %6d %s" "<[format %.3f $opt(-min)]" $hist(below) \
            [string repeat "#" [expr {int($hist(below) * $opt(-width) / $maxc)}]]]
        unset hist(below)
    }
    foreach k [lsort -integer [array names hist]] {
        set lo [expr {$k * $opt(-bin)}]
        set bar [string repeat "#" [expr {int($hist($k) * $opt(-width) / $maxc)}]]
        puts [format "%10s %6d %s" [format %.3f $lo] $hist($k) $bar]
    }
}

```

Bins are in slack units. `-bin 0.020` with ns units gives 20 ps bins. Endpoints with slack below `-min` are collected into one "below" row so a single deep outlier does not stretch the chart.

4.4 Worst slack per launch/capture clock pair

A clock pair matrix is the first thing to look at when a design has many clocks. A negative slack on a pair like CLK_A -> CLK_B where no synchronous relationship exists means a missing false path or clock group, not a timing problem.

```

#-----
# File      : 04_worst_per_clock_pair.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Worst slack and violator count for every launch/capture pair
# Tool      : PrimeTime (pt_shell)
# Usage     : source 04_worst_per_clock_pair.tcl ; clock_pair_matrix
#-----

proc clock_pair_matrix {{delay_type max} {max_paths 50000}} {
    set paths [get_timing_paths -delay_type $delay_type -nworst 1 \

```

```

        -max_paths $max_paths -slack_lesser_than 1e9]
array set worst {}
array set nviol {}
array set npath {}
foreach_in_collection p $paths {
    set s [get_attribute $p slack]
    if {[string is double -strict $s]} { continue }
    set l [get_object_name [get_attribute -quiet $p startpoint_clock]]
    set c [get_object_name [get_attribute -quiet $p endpoint_clock]]
    if {$l eq ""} { set l "(none)" }
    if {$c eq ""} { set c "(none)" }
    set key "$l,$c"
    if {[info exists worst($key)] || $s < $worst($key)} { set worst($key) $s }
    if {[info exists npath($key)]} { set npath($key) 0; set nviol($key) 0 }
    incr npath($key)
    if {$s < 0} { incr nviol($key) }
}
set rows {}
foreach key [array names worst] {
    lappend rows [list $key $worst($key) $nviol($key) $npath($key)]
}
set rows [lsort -real -index 1 $rows]
puts [format "%-30s %-30s %9s %8s %8s" LAUNCH CAPTURE WORST NVIOL NPATH]
foreach r $rows {
    lassign $r key w v n
    lassign [split $key ","] l c
    puts [format "%-30s %-30s %9.3f %8d %8d" $l $c $w $v $n]
}
return $rows
}

```

startpoint_clock and endpoint_clock are attributes of the path, not of the pins, so they reflect the clock the timer actually used for that path, including generated clocks and case-analysis-selected clocks.

4.5 reg2reg, in2reg, reg2out, in2out

I/O paths often fail because the I/O constraint is wrong, not because the logic is slow. Separating them from register-to-register paths lets you hand each list to the right person.

```

#-----
# File      : 04_path_classify.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Classify failing paths as reg2reg / in2reg / reg2out / in2out
# Tool      : PrimeTime (pt_shell)
# Usage     : source 04_path_classify.tcl ; classify_violators
#-----

proc path_class {path} {
    set sp [get_attribute $path startpoint]
    set ep [get_attribute $path endpoint]
    set sc [get_attribute $sp object_class]
    set ec [get_attribute $ep object_class]
    set s [expr {$sc eq "port" ? "in" : "reg"}]
    set e [expr {$ec eq "port" ? "out" : "reg"}]
    # macro pins are still "reg" class here; refine with is_hierarchical
    # on the parent cell when memories need their own bucket
    if {$sc eq "pin"} {
        set cell [get_cells -of_objects $sp]
        if {[get_attribute -quiet $cell is_black_box] eq "true"} { set s "macro" }
    }
    if {$ec eq "pin"} {
        set cell [get_cells -of_objects $ep]
        if {[get_attribute -quiet $cell is_black_box] eq "true"} { set e "macro" }
    }
}

```

```

    return "${s}2${e}"
}

proc classify_violators {{delay_type max} {max_paths 20000}} {
    set paths [get_timing_paths -delay_type $delay_type -nworst 1 \
        -max_paths $max_paths -slack_lesser_than 0]
    array set n {}
    array set w {}
    array set tns {}
    foreach_in_collection p $paths {
        set cls [path_class $p]
        set s [get_attribute $p slack]
        if ![info exists n($cls)] { set n($cls) 0; set w($cls) 0; set tns($cls) 0.0 }
        incr n($cls)
        set tns($cls) [expr {$tns($cls) + $s}]
        if {$s < $w($cls)} { set w($cls) $s }
    }
    puts [format "%-12s %8s %10s %12s" CLASS COUNT WNS TNS]
    foreach cls [lsort [array names n]] {
        puts [format "%-12s %8d %10.3f %12.3f" $cls $n($cls) $w($cls) $tns($cls)]
    }
}

```

The macro bucket relies on `is_black_box`, which is true for cells whose timing comes from a `.lib` with no internal netlist (memories, IP hard macros). Adjust to `ref_name =~` patterns if your library marks macros differently.

4.6 Where the delay is inside a path

For one path, the question is which stage is slow. `biggest_stages` walks the `points` collection, computes the increment between consecutive points, and prints the largest.

```

#-----
# File      : 04_path_points.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Walk the points of a path to find the largest stage delays
# Tool      : PrimeTime (pt_shell)
# Usage     : source 04_path_points.tcl ; biggest_stages [get_timing_paths] 5
#-----

# Prints the N stages (point-to-point increments) with the largest delay
# in a path, with the cell ref_name and transition at that point.
proc biggest_stages {path {top_n 5}} {
    set prev_arr ""
    set rows {}
    foreach_in_collection pt [get_attribute $path points] {
        set obj [get_attribute $pt object]
        set arr [get_attribute $pt arrival]
        set tran [get_attribute -quiet $pt transition]
        set name [get_object_name $obj]
        set ref ""
        if {[get_attribute $obj object_class] eq "pin"} {
            set ref [get_attribute -quiet [get_cells -of_objects $obj] ref_name]
        }
        if {$prev_arr ne "" && [string is double -strict $arr]} {
            lappend rows [list [expr {$arr - $prev_arr}] $name $ref $tran]
        }
        set prev_arr $arr
    }
    set rows [lsort -real -decreasing -index 0 $rows]
    puts [format "%8s %-50s %-20s %8s" INCR POINT REF TRAN]
    foreach r [lrange $rows 0 [expr {$top_n - 1}]] {
        lassign $r d n ref t
        puts [format "%8.3f %-50s %-20s %8s" $d $n $ref $t]
    }
    return $rows
}

```

```

# Sum of cell delay vs net delay along a path, approximated as:
# cell stage = increment where point is an output pin
# net stage = increment where point is an input pin
proc cell_net_split {path} {
    set prev_arr ""
    set cell 0.0
    set net 0.0
    foreach_in_collection pt [get_attribute $path points] {
        set obj [get_attribute $pt object]
        set arr [get_attribute $pt arrival]
        if {$prev_arr ne "" && [string is double -strict $arr]} {
            set d [expr {$arr - $prev_arr}]
            set dir [get_attribute -quiet $obj direction]
            if {$dir eq "in"} { set net [expr {$net + $d}] } else { set cell [expr {$cell + $d}] }
        }
        set prev_arr $arr
    }
    return [list $cell $net]
}

```

`cell_net_split` uses the direction of the pin at each point to attribute an increment to cell delay (increment ending at an output pin) or net delay (increment ending at an input pin). It is an approximation that ignores clock network points, which is acceptable for data path review.

A path where net delay dominates is a placement or routing problem; a path where one cell contributes 30% is a sizing problem. Chapter 8 uses the same walk to pick ECO candidates.

Chapter 5. Clock analysis

Half of all "timing violations" found in the first pass of a new block are clock definition problems: a generated clock with the wrong divide ratio, a master clock that never reaches its generated clock's source pin, a clock defined on a port that also has an input delay. These scripts find them before anyone spends time on the data path.

5.1 Clock inventory

```

#-----
# File      : 05_clock_summary.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Clock inventory: period, source, generated master, register count
# Tool      : PrimeTime (pt_shell)
# Usage     : source 05_clock_summary.tcl ; clock_summary
#-----

proc clock_summary {} {
    puts [format "%-28s %8s %-10s %-20s %8s %s" CLOCK PERIOD GEN MASTER NREGS SOURCE]
    set rows {}
    foreach_in_collection c [sort_collection [get_clocks] full_name] {
        set name [get_object_name $c]
        set period [get_attribute $c period]
        set gen [get_attribute -quiet $c is_generated]
        set master ""
        if {$gen eq "true"} {
            set master [get_object_name [get_attribute -quiet $c master_clock]]
        }
        set srcs [get_attribute -quiet $c sources]
    }
}

```

```

    set src ""
    if {$srcs ne ""} { set src [get_object_name [index_collection $srcs 0]] }
    set regs [all_registers -clock_pins -clock $c]
    set nregs [sizeof_collection $regs]
    puts [format "%-28s %8s %-10s %-20s %8d %s" $name $period $gen $master $nregs $src]
    lappend rows [list $name $period $gen $master $nregs $src]
}
return $rows
}

# Clocks that drive zero registers: usually a constraint bug.
proc unused_clocks {} {
    set out {}
    foreach_in_collection c [get_clocks] {
        if {[sizeof_collection [all_registers -clock_pins -clock $c]] == 0} {
            lappend out [get_object_name $c]
        }
    }
    puts "clocks driving no registers: $out"
    return $out
}

```

The register count column is the important one. A clock with zero registers is either a virtual clock (fine, if intended for I/O constraints) or a clock definition on the wrong pin. A generated clock with far fewer registers than its master often has the wrong `-source`.

`all_registers -clock -clock_pins` returns the clock pins of registers clocked by that clock, which is the only reliable way to count them. Tracing the clock network with `all_fanout` would include ICGs and clock buffers.

5.2 Latency and skew

`report_clock_timing -type skew` exists, but its output is not easy to post-process and it does not tell you which endpoint has the extreme latency. The path attributes `startpoint_clock_latency` and `endpoint_clock_latency` are available on every path object and give the latency the timer used for that specific path.

```

#-----
# File      : 05_latency_skew.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Clock latency statistics and skew from path attributes
# Tool      : PrimeTime (pt_shell)
# Usage     : source 05_latency_skew.tcl ; latency_stats CLK
#-----

# Min / max / mean capture-clock latency per clock, from the worst path
# to each endpoint. Uses endpoint_clock_latency on timing_path objects.
proc latency_stats {{clk *}} {max_paths 50000} {
    set paths [get_timing_paths -to [all_registers -clock_pins -clock [get_clocks $clk]] \
        -nworst 1 -max_paths $max_paths -slack_lesser_than 1e9]

    array set mn {}
    array set mx {}
    array set sum {}
    array set n {}
    array set mx_ep {}
    array set mn_ep {}
    foreach_in_collection p $paths {
        set c [get_object_name [get_attribute -quiet $p endpoint_clock]]
        set l [get_attribute -quiet $p endpoint_clock_latency]
        if {$c eq "" || ![string is double -strict $l]} { continue }
        if {[info exists n($c)]} {
            set n($c) 0; set sum($c) 0.0; set mn($c) $l; set mx($c) $l
            set mn_ep($c) ""; set mx_ep($c) ""
        }
    }
}

```

```

        incr n($c)
        set sum($c) [expr {$sum($c) + $l}]
        if {$l < $mn($c)} { set mn($c) $l; set mn_ep($c) [get_object_name [get_attribute $p
endpoint]] }
        if {$l > $mx($c)} { set mx($c) $l; set mx_ep($c) [get_object_name [get_attribute $p
endpoint]] }
    }
    puts [format "%-24s %6s %8s %8s %8s %8s" CLOCK N MIN MAX MEAN SKEW]
    foreach c [lsort [array names n]] {
        puts [format "%-24s %6d %8.3f %8.3f %8.3f %8.3f" $c $n($c) $mn($c) $mx($c) [
[expr {$sum($c) / $n($c)}] [expr {$mx($c) - $mn($c)}]]
        puts "    min endpoint: $mn_ep($c)"
        puts "    max endpoint: $mx_ep($c)"
    }
}

# Local skew on failing paths: launch latency minus capture latency.
# Positive value means the launch clock arrives later (hurts setup).
proc skew_on_violators {{top_n 20}} {
    set paths [get_timing_paths -delay_type max -nworst 1 -max_paths 5000 -slack_lesser_than 0]
    set rows {}
    foreach_in_collection p $paths {
        set ll [get_attribute -quiet $p startpoint_clock_latency]
        set cl [get_attribute -quiet $p endpoint_clock_latency]
        set crpr [get_attribute -quiet $p common_path_pessimism]
        if {[string is double -strict $ll] || ![string is double -strict $cl]} { continue }
        if {[string is double -strict $crpr]} { set crpr 0.0 }
        lappend rows [list [expr {$ll - $cl}] [get_attribute $p slack] $ll $cl $crpr [
[get_object_name [get_attribute $p endpoint]]]]
    }
    set rows [lsort -real -decreasing -index 0 $rows]
    puts [format "%8s %8s %8s %8s %8s %s" SKEW SLACK LAUNCH CAPT CRPR ENDPOINT]
    foreach r [lrange $rows 0 [expr {$top_n - 1}]] {
        lassign $r sk s ll cl crpr ep
        puts [format "%8.3f %8.3f %8.3f %8.3f %8.3f %s" $sk $s $ll $cl $crpr $ep]
    }
    return $rows
}

```

`latency_stats` reports the spread of capture latency per clock with the endpoints at each extreme. A spread larger than a clock buffer stage on a balanced tree points at an unbalanced branch, a hold-fix insertion on a clock line, or a wrong `set_clock_latency` override.

`skew_on_violators` shows how much of each failing path's slack is explained by skew. A path with skew of 150 ps and slack of minus 120 ps is a clock tree problem; the data path is fine.

`common_path_pessimism` is the CRPR credit the timer applied. If it is zero on a path where launch and capture share most of the tree, `timing_remove_clock_reconvergence_pessimism` is probably off.

5.3 Generated clock sanity

```

#-----
# File      : 05_generated_clock_check.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Sanity checks on generated clocks and clock sources
# Tool      : PrimeTime (pt_shell)
# Usage     : source 05_generated_clock_check.tcl ; generated_clock_check
#-----

# Flags:
# - generated clock whose master drives no registers itself

```

```

# - generated clock whose source pin has no master clock on it
# - two clocks defined on the same source pin (usually intended, but list)
proc generated_clock_check {} {
    array set by_src {}
    set issues 0
    foreach_in_collection c [get_clocks] {
        set name [get_object_name $c]
        set srcs [get_attribute -quiet $c sources]
        foreach_in_collection s $srcs {
            set sn [get_object_name $s]
            lappend by_src($sn) $name
        }
        if {[get_attribute -quiet $c is_generated] ne "true"} { continue }
        set master [get_attribute -quiet $c master_clock]
        if {$master eq ""} {
            puts "ISSUE $name : generated clock has no master"
            incr issues
            continue
        }
        set mname [get_object_name $master]
        if {$srcs ne ""} {
            set src0 [index_collection $srcs 0]
            set clks_on_src [get_attribute -quiet $src0 clocks]
            set found 0
            foreach_in_collection k $clks_on_src {
                if {[get_object_name $k] eq $mname} { set found 1 }
            }
            if {!$found} {
                puts "ISSUE $name : master $mname does not reach source [get_object_name $src0]"
                incr issues
            }
        }
    }
    foreach sn [array names by_src] {
        if {[llength $by_src($sn)] > 1} {
            puts "NOTE $sn : multiple clocks defined here: $by_src($sn)"
        }
    }
    puts "generated clock issues: $issues"
    return $issues
}

```

The check reads the `clocks` attribute on the generated clock's source pin. If the master clock is not in that list, the master does not propagate to the source, and PrimeTime will report the generated clock with zero latency or no timing at all. The usual cause is a `set_case_analysis` on a mux select that blocks the master, or a `set_disable_timing` on the divider.

Multiple clocks on one source pin are listed as notes rather than issues; they are legitimate when the pin carries a mux of two clocks that are timed in the same mode, but each such case should be intentional.

5.4 Constraint commands this chapter assumes

Command	What the scripts expect
<code>create_clock -period P -name N [get_ports ...]</code>	<code>sources</code> attribute holds the port
<code>create_generated_clock -source S -divide_by N -master_clock M</code>	<code>master_clock</code> , <code>is_generated</code> set
<code>set_propagated_clock [all_clocks]</code>	latency attributes are real, not ideal

Command	What the scripts expect
<code>set_clock_groups -asynchronous</code>	pair matrix rows for those pairs are absent
<code>set_clock_uncertainty</code>	included in <code>required</code> , not in <code>latency</code>

Chapter 6. Constraint audit

An SDC that loads without errors is not a correct SDC. The tests in this chapter measure what the constraints cover and what they leave out. Run them on every SDC delivery, before any timing number is discussed.

6.1 Endpoints with no timing check

An endpoint with `max_slack` of `INFINITY` is not timed. `report_analysis_coverage` counts them; this script names them and rolls them up by block, so a list of 8,000 pins becomes "u_dma has 7,900 of them", which points at a missing clock rather than 7,900 individual problems.

```
#-----
# File      : 06_unconstrained_endpoints.tcl
# Author    : youbumkim@gmail.com
# Purpose   : List register data pins and output ports with no timing check
# Tool      : PrimeTime (pt_shell)
# Usage     : source 06_unconstrained_endpoints.tcl ; unconstrained_endpoints out.rpt
#-----

# Requires: set_app_var timing_save_pin_arrival_and_slack true (before update_timing)
# A pin whose max_slack is INFINITY has no setup check reaching it.
proc unconstrained_endpoints {{out ""}} {
    set eps [add_to_collection [all_registers -data_pins] [all_outputs]]
    set rows {}
    foreach_in_collection e $eps {
        set s [get_attribute -quiet $e max_slack]
        if {$s eq "" || $s eq "INFINITY"} {
            set cls [get_attribute $e object_class]
            set ref ""
            if {$cls eq "pin"} {
                set ref [get_attribute -quiet [get_cells -of_objects $e] ref_name]
            }
            lappend rows [list [get_object_name $e] $cls $ref]
        }
    }
    set rows [lsort -index 0 $rows]
    if {$out ne ""} {
        set fh [open $out w]
        foreach r $rows { puts $fh [join $r "\t"] }
        close $fh
    }
    puts "unconstrained endpoints: [llength $rows] of [sizeof_collection $eps]"
    return $rows
}

# Roll the list up by hierarchy so a 10,000-line list becomes 20 lines.
proc unconstrained_by_block {{depth 2}} {
    array set n {}
    foreach r [unconstrained_endpoints] {
        set name [lindex $r 0]
        set parts [split $name "/"]
    }
}
```

```

        set key [join [lrange $parts 0 [expr {$depth - 1}]] "/"]
        if {[info exists n($key)]} { incr n($key) } else { set n($key) 1 }
    }
    set rows {}
    foreach k [array names n] { lappend rows [list $k $n($k)] }
    set rows [lsort -integer -decreasing -index 1 $rows]
    puts [format "%-50s %8s" BLOCK UNCONSTRAINED]
    foreach r $rows { puts [format "%-50s %8d" [lindex $r 0] [lindex $r 1]] }
    return $rows
}

```

The proc requires `timing_save_pin_arrival_and_slack true` set before `update_timing`, as described in Chapter 2. Without it, `max_slack` is empty on every pin and the script reports every endpoint as unconstrained.

6.2 Exceptions that apply to nothing

`report_exceptions -ignored` lists `set_false_path`, `set_multicycle_path`, `set_max_delay` and `set_min_delay` commands that matched no path. Each one is either a typo in an object name, a hierarchy that was renamed, or an exception that was never needed. All three are worth knowing about.

```

#-----
# File      : 06_exception_audit.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Audit false paths, multicycle paths and ignored exceptions
# Tool      : PrimeTime (pt_shell)
# Usage     : source 06_exception_audit.tcl ; exception_audit rpt_dir
#-----

proc exception_audit {rpt_dir} {
    file mkdir $rpt_dir
    # Exceptions that do not apply to any path: wasted or wrong.
    redirect -file $rpt_dir/exceptions_ignored.rpt {
        report_exceptions -ignored
    }
    # Every exception, with the count of paths it dominates.
    redirect -file $rpt_dir/exceptions_all.rpt {
        report_exceptions
    }
    # Coverage of every check type.
    redirect -file $rpt_dir/coverage.rpt {
        report_analysis_coverage -status_details {untested violated}
    }

    # Count exception commands in each category from the full report.
    set fh [open $rpt_dir/exceptions_all.rpt r]
    array set n {false_path 0 multicycle_path 0 max_delay 0 min_delay 0}
    while {[gets $fh line] >= 0} {
        if {[regexp {\s*(false_path|multicycle_path|max_delay|min_delay)} $line -> t]} {
            incr n($t)
        }
    }
    close $fh
    puts [format "%-20s %8s" EXCEPTION COUNT]
    foreach t {false_path multicycle_path max_delay min_delay} {
        puts [format "%-20s %8d" $t $n($t)]
    }
    # Ignored count
    set fh [open $rpt_dir/exceptions_ignored.rpt r]
    set ignored 0
    while {[gets $fh line] >= 0} {
        if {[regexp {\s*(false_path|multicycle_path|max_delay|min_delay)} $line]} { incr
ignored }
    }
}

```

```

    close $fh
    puts "ignored exceptions : $ignored    (see $rpt_dir/exceptions_ignored.rpt)"
}

# Multicycle paths whose setup multiplier is set but hold multiplier is
# missing: the classic -setup 2 without -hold 1 mistake.
proc mcp_hold_check {} {
    redirect -variable txt { report_exceptions -nosplit }
    set setup_only 0
    foreach line [split $txt "\n"] {
        if {[regexp {multicycle_path} $line] && [regexp {setup} $line] && ![regexp {hold} $line]}
        {
            incr setup_only
            puts "    $line"
        }
    }
    puts "multicycle setup-only entries (verify hold): $setup_only"
    return $setup_only
}

```

`mcp_hold_check` looks for the most common multicycle mistake: `set_multicycle_path -setup 2` without a matching `-hold 1`. PrimeTime moves the hold check to the same edge as the setup check, one cycle later, and the path then needs a full cycle of hold margin. The proc lists multicycle entries that mention setup but not hold; review each one.

6.3 I/O constraint coverage

```

#-----
# File      : 06_io_constraint_coverage.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Ports missing input/output delay, drive, or load
# Tool      : PrimeTime (pt_shell)
# Usage     : source 06_io_constraint_coverage.tcl ; io_coverage
#-----

proc io_coverage {} {
    set no_in_delay {}
    set no_out_delay {}
    set no_drive {}
    set no_load {}
    set clocks_src {}
    # ports that are clock sources are excluded from delay checks
    foreach_in_collection c [get_clocks] {
        foreach_in_collection s [get_attribute -quiet $c sources] {
            if {[get_attribute $s object_class] eq "port"} {
                lappend clocks_src [get_object_name $s]
            }
        }
    }
    foreach_in_collection p [all_inputs] {
        set n [get_object_name $p]
        if {[lsearch -exact $clocks_src $n] >= 0} { continue }
        # max_slack INFINITY on an input means nothing launches from it
        set s [get_attribute -quiet $p max_slack]
        if {$s eq "" || $s eq "INFINITY"} { lappend no_in_delay $n }
        set dr [get_attribute -quiet $p driving_cell_rise_max]
        set tr [get_attribute -quiet $p input_transition_max_rise]
        if {$dr eq "" && $tr eq ""} { lappend no_drive $n }
    }
    foreach_in_collection p [all_outputs] {
        set n [get_object_name $p]
        set s [get_attribute -quiet $p max_slack]
        if {$s eq "" || $s eq "INFINITY"} { lappend no_out_delay $n }
        set ld [get_attribute -quiet $p pin_capacitance_max]
        if {$ld eq "" || $ld == 0} { lappend no_load $n }
    }
    puts "inputs without arrival/delay : [llength $no_in_delay]"
}

```

```

puts "inputs without drive/transition: [llength $no_drive]"
puts "outputs without required/delay: [llength $no_out_delay]"
puts "outputs without load          : [llength $no_load]"
return [list $no_in_delay $no_drive $no_out_delay $no_load]
}

```

The four lists returned are:

1. Input ports with no path timed from them (no `set_input_delay`, or the delay refers to a clock that does not exist).
2. Input ports with no `set_driving_cell` or `set_input_transition`, so the timer uses a zero transition and the first stage is optimistic.
3. Output ports with no path timed to them.
4. Output ports with no `set_load`, so the driver sees no capacitance and is optimistic.

Clock source ports are excluded from the delay checks. The port attribute names for driving cell and input transition vary between releases; if both come back empty on a port you know is constrained, check `list_attributes -application -class port`.

6.4 Case analysis and disabled arcs

`set_case_analysis` is how a mode is selected: scan enable low, test mode off, a clock mux select. A wrong case value silently removes logic from analysis. The script writes the case and disable reports and adds a specific check: register clock pins held constant by case analysis, which means those registers are not timed at all.

```

#-----
# File      : 06_case_and_disable.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Report case analysis and disabled arcs with their origin
# Tool      : PrimeTime (pt_shell)
# Usage     : source 06_case_and_disable.tcl ; case_disable_report rpt_dir
#-----

proc case_disable_report {rpt_dir} {
    file mkdir $rpt_dir
    redirect -file $rpt_dir/case_analysis.rpt { report_case_analysis -nosplit }
    redirect -file $rpt_dir/disable_timing.rpt { report_disable_timing -nosplit }

    # Registers whose clock pin is held constant by case analysis:
    # they will never be timed, and that is sometimes not intended.
    set held {}
    foreach_in_collection pin [all_registers -clock_pins] {
        set v [get_attribute -quiet $pin case_value]
        if {$v ne ""} { lappend held [get_object_name $pin] }
    }
    puts "register clock pins with constant case value: [llength $held]"
    set fh [open $rpt_dir/case_held_clock_pins.rpt w]
    foreach h $held { puts $fh $h }
    close $fh

    # Disabled arcs on user cells (not library-defined) come from
    # set_disable_timing in the SDC; count them per cell type.
    array set n {}
    redirect -variable txt { report_disable_timing -nosplit }
    foreach line [split $txt "\n"] {
        if {[regexp {\^S*(S+)S+S+S+S+S+(user|case-analysis|constant|loop-breaking)}
$line -> cell why]} {
            if {[info exists n($why)]} { incr n($why) } else { set n($why) 1 }
        }
    }
}

```

```

    foreach why [lsort [array names n]] {
        puts [format "    disabled arcs (%-16s) : %d" $why $n($why)]
    }
}

```

The disabled-arc counts by origin separate library-defined disables (normal) from user set_disable_timing (review each) and from loop breaking (the timer broke a combinational loop; the choice of arc is arbitrary and may hide a real path).

Chapter 7. Violation triage

Triage is the step between "there are 4,000 violations" and "there are three problems". The scripts here group violators so that each group has one owner and one likely fix.

7.1 Bucket by block, clock pair and severity

```

#-----
# File      : 07_bucket_violators.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Group setup/hold violators by block, clock pair, and severity
# Tool      : PrimeTime (pt_shell)
# Usage     : source 07_bucket_violators.tcl ; bucket_violators -depth 2 -out v.csv
#-----

proc bucket_violators {args} {
    array set opt {-depth 2 -delay_type max -max_paths 50000 -out "" -group {}}
    array set opt $args
    set cmd [list get_timing_paths -delay_type $opt(-delay_type) -nworst 1 \
        -max_paths $opt(-max_paths) -slack_lesser_than 0]
    if {$opt(-group) ne ""} { lappend cmd -group $opt(-group) }
    set paths [eval $cmd]

    array set cnt {}
    array set wns {}
    array set tns {}
    set rows {}
    foreach_in_collection p $paths {
        set s [get_attribute $p slack]
        set sp [get_object_name [get_attribute $p startpoint]]
        set ep [get_object_name [get_attribute $p endpoint]]
        set lc [get_object_name [get_attribute -quiet $p startpoint_clock]]
        set cc [get_object_name [get_attribute -quiet $p endpoint_clock]]
        set sblk [join [lrange [split $sp "/"] 0 [expr {$opt(-depth) - 1}]] "/"]
        set eblk [join [lrange [split $ep "/"] 0 [expr {$opt(-depth) - 1}]] "/"]
        if {$s <= -0.100} { set sev S1 } elseif {$s <= -0.030} { set sev S2 } else { set sev S3 }
        set key "$sblk|$eblk|$lc|$cc"
        if {![info exists cnt($key)]} { set cnt($key) 0; set wns($key) 0; set tns($key) 0.0 }
        incr cnt($key)
        set tns($key) [expr {$tns($key) + $s}]
        if {$s < $wns($key)} { set wns($key) $s }
        lappend rows [list $s $sev $sblk $eblk $lc $cc $sp $ep]
    }

    puts [format "%-28s %-28s %-14s %-14s %6s %9s %10s" FROM_BLOCK TO_BLOCK LAUNCH CAPTURE N WNS
TNS]
    set keys {}
    foreach k [array names cnt] { lappend keys [list $k $tns($k)] }
    foreach kk [lsort -real -index 1 $keys] {
        set k [lindex $kk 0]
        lassign [split $k "|"] a b l c
        puts [format "%-28s %-28s %-14s %-14s %6d %9.3f %10.3f" $a $b $l $c $cnt($k) $wns($k)
$tns($k)]
    }
}

```

```

    if {$opt(-out) ne ""} {
        set fh [open $opt(-out) w]
        puts $fh
        "slack,severity,from_block,to_block,launch_clock,capture_clock,startpoint,endpoint"
        foreach r [lsort -real -index 0 $rows] { puts $fh [join $r ","] }
        close $fh
        puts "wrote [llength $rows] violators to $opt(-out)"
    }
    return $rows
}

```

The key is `from_block|to_block|launch_clock|capture_clock` at the hierarchy depth you choose. Depth 2 on a typical SoC gives subsystem-level buckets; depth 3 gives block-level. The table is sorted by TNS so the bucket that contributes the most total negative slack is first.

Severity bands are fixed at minus 100 ps and minus 30 ps in the script. Those are reasonable at a 1 GHz clock; scale them for your frequency.

The CSV written with `-out` has one row per violating endpoint and is the input for the run-to-run diff in Chapter 9 and the multi-corner merge in Chapter 10.

7.2 Cells that many failing paths share

When 400 failing paths all go through one slow mux, fixing that mux fixes 400 paths.

`common_cells_in_violators` counts, for each leaf cell, how many distinct failing paths pass through it.

```

#-----
# File      : 07_common_cells.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Cells that appear in the most failing paths (fix these first)
# Tool      : PrimeTime (pt_shell)
# Usage     : source 07_common_cells.tcl ; common_cells_in_violators 30
#-----

# Counts how many distinct failing paths pass through each leaf cell.
# A cell on 500 failing paths is a better ECO target than one on 2.
proc common_cells_in_violators {{top_n 30} {max_paths 2000} {delay_type max}} {
    set paths [get_timing_paths -delay_type $delay_type -nworst 1 \
        -max_paths $max_paths -slack_lesser_than 0]
    array set n {}
    array set worst {}
    foreach_in_collection p $paths {
        set s [get_attribute $p slack]
        array set seen {}
        foreach_in_collection pt [get_attribute $p points] {
            set obj [get_attribute $pt object]
            if {[get_attribute $obj object_class] ne "pin"} { continue }
            set cell [get_object_name [get_cells -of_objects $obj]]
            if {[info exists seen($cell)]} { continue }
            set seen($cell) 1
            if {[info exists n($cell)]} { incr n($cell) } else { set n($cell) 0; set worst($cell)
0 }
            if {$s < $worst($cell)} { set worst($cell) $s }
        }
        array unset seen
    }
    set rows {}
    foreach c [array names n] {
        lappend rows [list $c $n($c) $worst($c) [get_attribute [get_cells $c] ref_name]]
    }
    set rows [lsort -integer -decreasing -index 1 $rows]
}

```

```

puts [format "%-55s %6s %9s %s" CELL NPATH WORST REF]
foreach r [lrange $rows 0 [expr {$top_n - 1}]] {
    lassign $r c cnt w ref
    puts [format "%-55s %6d %9.3f %s" $c $cnt $w $ref]
}
return $rows
}

```

Walking the points of 2,000 paths takes a few seconds on a block. On a full chip, restrict max_paths or add -group. The seen array makes sure a cell is counted once per path even if the path enters and exits the cell (input pin and output pin are separate points).

The output is ranked by path count. Cross-check the top entries against worst: a cell on many paths but with a worst slack of minus 5 ps is a marginal fix, while a cell on 50 paths at minus 200 ps is a real target.

7.3 Design rule violations

Max transition and max capacitance violations often explain setup violations in the same region, and they must be clean for signoff regardless. drv_table reads the actual and limit values from pin and net attributes and prints a sortable table.

```

#-----
# File      : 07_drv_summary.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Max transition / max capacitance violators as a table
# Tool      : PrimeTime (pt_shell)
# Usage     : source 07_drv_summary.tcl ; drv_table -out drv.csv
#-----

# Pin-attribute based DRV table. Attribute names differ slightly across
# releases; verify with: list_attributes -application -class pin
proc drv_table {args} {
    array set opt {-out "" -top 50}
    array set opt $args
    set rows {}
    set pins [get_pins -hierarchical -filter "direction == out && is_hierarchical == false"]
    foreach_in_collection p $pins {
        set lim [get_attribute -quiet $p max_transition]
        if {[string is double -strict $lim]} { continue }
        set tr [get_attribute -quiet $p actual_rise_transition_max]
        set tf [get_attribute -quiet $p actual_fall_transition_max]
        set t $tr
        if {[string is double -strict $tf] && (![string is double -strict $t] || $tf > $t)} { set
t $tf }
        if {[string is double -strict $t] && $t > $lim} {
            lappend rows [list max_transition [get_object_name $p] $t $lim [expr {$lim - $t}]]
        }
        set clim [get_attribute -quiet $p max_capacitance]
        if {[string is double -strict $clim]} {
            set net [get_nets -quiet -of_objects $p]
            if {$net ne ""} {
                set c [get_attribute -quiet $net total_capacitance_max]
                if {[string is double -strict $c] && $c > $clim} {
                    lappend rows [list max_capacitance [get_object_name $p] $c $clim [expr {$clim
- $c}]]
                }
            }
        }
    }
    set rows [lsort -real -index 4 $rows]
    puts [format "%-16s %-55s %9s %9s %9s" CHECK PIN ACTUAL LIMIT SLACK]
    foreach r [lrange $rows 0 [expr {$opt(-top) - 1}]] {
        lassign $r chk pin a l s
        puts [format "%-16s %-55s %9.4f %9.4f %9.4f" $chk $pin $a $l $s]
    }
}

```

```

    }
    puts "total DRV violators: [llength $rows]"
    if {$opt(-out) ne ""} {
        set fh [open $opt(-out) w]
        puts $fh "check,pin,actual,limit,slack"
        foreach r $rows { puts $fh [join $r ","] }
        close $fh
    }
    return $rows
}

# Fallback that does not depend on attribute names: parse report_constraint.
proc drv_count_from_report {} {
    redirect -variable txt {
        report_constraint -all_violators -max_transition -max_capacitance -nosplit
    }
    set n_tran 0
    set n_cap 0
    set section ""
    foreach line [split $txt "\n"] {
        if {[regexp {max_transition} $line]} { set section tran }
        if {[regexp {max_capacitance} $line]} { set section cap }
        if {[regexp {(VIOLATED)} $line]} {
            if {$section eq "tran"} { incr n_tran } else { incr n_cap }
        }
    }
    puts "max_transition violators : $n_tran"
    puts "max_capacitance violators: $n_cap"
    return [list $n_tran $n_cap]
}

```

Attribute names for actual transition (`actual_rise_transition_max`, `actual_fall_transition_max`) and net capacitance (`total_capacitance_max`) are the ones current releases use. `drv_count_from_report` is the release-independent fallback: it counts (VIOLATED) markers in `report_constraint -all_violators` output and needs no attribute names at all.

7.4 What to do with each bucket

Bucket pattern	Likely cause	First action
One clock pair, many blocks, slack near zero	Clock uncertainty or derate change	Compare env report (2.2) with last passing run
Async clock pair with violations	Missing <code>set_clock_groups</code> or false path	Constraint owner
One block, one clock, deep negative	Logic depth	PBA report (9.4), then RTL or synthesis
in2reg or reg2out only	I/O delay values	Check 6.3 coverage, then interface spec
Many endpoints sharing one cell in 7.2	Weak driver or high fanout	ECO sizing (8.1)
DRV violators in same region	Long net or missing buffer	<code>fix_eco_drc</code> (8.5) before setup ECO

Chapter 8. ECO scripting

PrimeTime can size cells and insert buffers in its own netlist view, re-time incrementally, and write the changes out for the implementation tool. This chapter covers the three ECO tasks that are worth scripting: choosing what to change, applying the change with a check that it helped, and exporting it.

All scripts in this chapter need the ECO capability. `size_cell`, `insert_buffer`, `remove_buffer`, `write_changes`, `fix_eco_timing` and `fix_eco_drc` are not available without it.

8.1 Choosing sizing candidates

Automatic fixers pick candidates by their own cost function. A scripted candidate list lets you apply your own rules: skip clock cells, skip delay cells, skip anything marked `dont_touch`, and rank by the number of failing paths that would benefit.

```
#-----
# File      : 08_eco_candidates.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Pick upsizing candidates on failing paths and rank them
# Tool      : PrimeTime (pt_shell)
# Usage     : source 08_eco_candidates.tcl ; eco_size_candidates -out cand.csv
#-----

# Candidate = leaf cell on a failing path whose output transition is the
# slowest on the path, that is not dont_touch, and that has a larger
# alternative lib cell available (same footprint check is left to the
# implementation tool).
proc eco_size_candidates {args} {
    array set opt {-max_paths 1000 -out "" -min_slack -1.0 -skip_ref {DLY* *CKBUF* *ICG*}}
    array set args
    set paths [get_timing_paths -delay_type max -nworst 1 -max_paths $opt(-max_paths) \
        -slack_lesser_than 0 -slack_greater_than $opt(-min_slack)]

    array set score {}
    array set worst {}
    array set ref {}
    foreach_in_collection p $paths {
        set s [get_attribute $p slack]
        set best_tran 0
        set best_cell ""
        foreach_in_collection pt [get_attribute $p points] {
            set obj [get_attribute $pt object]
            if {[get_attribute $obj object_class] ne "pin"} { continue }
            if {[get_attribute -quiet $obj direction] ne "out"} { continue }
            set tr [get_attribute -quiet $pt transition]
            if {[string is double -strict $tr]} { continue }
            set cell [get_cells -of_objects $obj]
            if {[get_attribute -quiet $cell dont_touch] eq "true"} { continue }
            if {[get_attribute -quiet $cell is_sequential] eq "true"} { continue }
            set r [get_attribute $cell ref_name]
            set skip 0
            foreach g $opt(-skip_ref) { if {[string match $g $r]} { set skip 1 } }
            if {$skip} { continue }
            if {$tr > $best_tran} {
                set best_tran $tr
                set best_cell [get_object_name $cell]
                set best_ref $r
            }
        }
        if {$best_cell eq ""} { continue }
        if {[info exists score($best_cell)]} {
            set score($best_cell) 0; set worst($best_cell) 0; set ref($best_cell) $best_ref
        }
        incr score($best_cell)
        if {$s < $worst($best_cell)} { set worst($best_cell) $s }
    }
}
```

```

set rows {}
foreach c [array names score] {
    set alts [get_alternative_lib_cells -quiet [get_cells $c]]
    set nalt 0
    if {$alts ne ""} { set nalt [sizeof_collection $alts] }
    lappend rows [list $c $ref($c) $score($c) $worst($c) $nalt]
}
set rows [lsort -integer -decreasing -index 2 $rows]
puts [format "%-55s %-18s %6s %9s %5s" CELL REF NPATH WORST NALT]
foreach r $rows {
    lassign $r c ref n w nalt
    puts [format "%-55s %-18s %6d %9.3f %5d" $c $ref $n $w $nalt]
}
if {$opt(-out) ne ""} {
    set fh [open $opt(-out) w]
    puts $fh "cell,ref_name,npath,worst_slack,n_alternatives"
    foreach r $rows { puts $fh [join $r ","] }
    close $fh
}
return $rows
}

```

For each failing path, the cell with the slowest output transition (excluding registers and skipped types) is nominated. A cell nominated by many paths rises to the top.

`get_alternative_lib_cells` returns the lib cells that are footprint- or function-compatible according to the library, so the last column says whether an upsizing is even possible.

The `-min_slack` option ignores paths worse than the given value. Paths at minus 500 ps will not be fixed by sizing one cell; leave them for a logic change and keep the ECO list to what sizing can fix.

8.2 Try, measure, keep or revert

```

#-----
# File      : 08_eco_upsize.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Try-and-keep upsizing loop with incremental timing checks
# Tool      : PrimeTime (pt_shell), requires ECO (size_cell) capability
# Usage     : source 08_eco_upsize.tcl ; eco_upsize_loop cand.csv -limit 200
#-----

# Reads candidate cells (first CSV column), tries the next drive strength
# from get_alternative_lib_cells, keeps the change only if the worst
# slack of the paths through the cell improves and no new DRV appears.
#
# Drive strength is inferred from a trailing X<n> or D<n> in the lib cell
# name. Adjust the regexp for your library naming.
proc drive_of {libcell name} {
    if {[regexp {[XD](\d+)[A-Z]*$} $libcell_name -> d]} { return $d }
    return 0
}

proc next_bigger_libcell {cell} {
    set cur [get_attribute $cell ref_name]
    set cur_d [drive_of $cur]
    set best ""
    set best_d 1e9
    foreach_in_collection alt [get_alternative_lib_cells -quiet $cell] {
        set n [get_attribute $alt base_name]
        set d [drive_of $n]
        if {$d > $cur_d && $d < $best_d} { set best_d $d; set best $alt }
    }
    return $best
}

```

```

proc worst_slack_through {cell} {
    set pins [get_pins -of_objects $cell -filter "direction == out"]
    set p [get_timing_paths -through $pins -nworst 1 -max_paths 1]
    if {$p eq ""} { return 1e9 }
    return [get_attribute $p slack]
}

proc eco_upsize_loop {csv args} {
    array set opt {-limit 100 -log eco_upsize.log}
    array set opt $args
    set fh [open $csv r]
    gets $fh ;# header
    set cells {}
    while {[gets $fh line] >= 0} {
        set c [lindex [split $line ","] 0]
        if {$c ne ""} { lappend cells $c }
    }
    close $fh
    set log [open $opt(-log) w]
    set n_try 0
    set n_keep 0
    foreach cname [lrange $cells 0 [expr {$opt(-limit) - 1}]] {
        set cell [get_cells -quiet $cname]
        if {$cell eq ""} { continue }
        set alt [next_bigger_libcell $cell]
        if {$alt eq ""} { puts $log "$cname : no larger alternative"; continue }
        set before [worst_slack_through $cell]
        set old_ref [get_attribute $cell ref_name]
        incr n_try
        size_cell $cell $alt
        update_timing
        set after [worst_slack_through $cell]
        if {$after > $before + 0.001} {
            incr n_keep
            puts $log [format "%s : %s -> %s slack %.3f -> %.3f KEEP" \
                $cname $old_ref [get_attribute $alt base_name] $before $after]
        } else {
            # revert: size back to the original lib cell
            set orig [get_lib_cells -quiet */$old_ref]
            size_cell $cell [index_collection $orig 0]
            update_timing
            puts $log [format "%s : %s -> %s slack %.3f -> %.3f REVERT" \
                $cname $old_ref [get_attribute $alt base_name] $before $after]
        }
    }
    close $log
    puts "tried $n_try, kept $n_keep (log: $opt(-log))"
    return $n_keep
}

```

The loop is deliberately conservative: one cell at a time, one `update_timing` per attempt, keep only if the worst slack through the cell improves by more than 1 ps. On a block with a few hundred candidates it runs in minutes because incremental `update_timing` after `size_cell` re-times only the affected cone.

The drive strength is read from the lib cell name with a regular expression that matches `x4`, `D8` and similar suffixes. Change `drive_of` to match your library's naming convention. If the library provides a drive strength attribute on lib cells, use that instead.

The revert path resolves the original lib cell with `get_lib_cells */$old_ref`, which assumes the base name is unique across linked libraries. Multi-library flows with the same cell name in two libraries should keep the full `lib/cell` name from `get_attribute $cell lib_cell` instead.

8.3 Hold fixing with a setup guard

Hold buffers inserted without checking setup slack are the classic way to turn a hold violation into a setup violation. `fix_hold_by_buffer` computes the number of buffers needed from the hold slack, estimates the cost with a per-buffer delay you supply, and skips the endpoint when the setup slack cannot absorb it.

```
#-----
# File      : 08_hold_buffer.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Insert hold buffers at endpoints with margin against setup
# Tool      : PrimeTime (pt_shell), requires ECO (insert_buffer) capability
# Usage     : source 08_hold_buffer.tcl ; fix_hold_by_buffer -buf lib/DLY2X1 -margin 0.02
#-----

# For every hold violator endpoint, insert a delay buffer on the data pin
# only if the setup slack at that endpoint can absorb the buffer delay.
proc fix_hold_by_buffer {args} {
    array set opt {-buf "" -margin 0.010 -max_paths 5000 -buf_delay 0.050 -log hold_fix.log}
    array set opt $args
    if {$opt{-buf} eq ""} { error "-buf lib/cell required" }
    set buf [get_lib_cells $opt{-buf}]
    set log [open $opt{-log} w]
    set paths [get_timing_paths -delay_type min -nworst 1 -max_paths $opt{-max_paths} \
        -slack_lesser_than 0]

    set n 0
    set skipped 0
    array set done {}
    foreach_in_collection p $paths {
        set ep [get_attribute $p endpoint]
        set en [get_object_name $ep]
        if {[info exists done($en)]} { continue }
        set done($en) 1
        set hold_slack [get_attribute $p slack]
        set setup_slack [get_attribute -quiet $ep max_slack]
        if {![string is double -strict $setup_slack]} { set setup_slack 1e9 }
        set need [expr {-1.0 * $hold_slack + $opt{-margin}}]
        set nbuf [expr {int(ceil($need / $opt{-buf_delay}))}]
        set cost [expr {$nbuf * $opt{-buf_delay}}]
        if {$setup_slack - $cost < $opt{-margin}} {
            puts $log [format "%s hold=%.3f setup=%.3f need=%d bufs SKIP (setup)" $en $hold_slack
                $setup_slack $nbuf]
            incr skipped
            continue
        }
        for {set i 0} {$i < $nbuf} {incr i} {
            insert_buffer $ep $buf
        }
        incr n
        puts $log [format "%s hold=%.3f setup=%.3f inserted %d x %s" $en $hold_slack $setup_slack
            $nbuf $opt{-buf}]
    }
    close $log
    update_timing
    puts "endpoints fixed: $n  skipped for setup: $skipped  (log $opt{-log})"
    return $n
}
```

`-buf_delay` is an estimate used only for the setup guard; the timer computes the real delay after insertion. Set it to the typical delay of the chosen delay cell in the hold corner. The proc processes each endpoint once (`done` array) because `get_timing_paths` may return several paths to the same endpoint when `-nworst 1` is combined with multiple launch clocks.

8.4 Export for the implementation tool

```
#-----
# File      : 08_write_eco.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Export ECO changes for the implementation tool and verify
# Tool      : PrimeTime (pt_shell)
# Usage     : source 08_write_eco.tcl ; export_eco eco_out
#-----

proc export_eco {dir} {
    file mkdir $dir
    # Native formats. Pick the one the P&R tool consumes.
    write_changes -format icctcl -output $dir/eco_icc2.tcl
    write_changes -format ptsh -output $dir/eco_pt.tcl
    # Innovus does not read icctcl; translate size_cell / insert_buffer.
    set fh [open $dir/eco_pt.tcl r]
    set oh [open $dir/eco_innovus.tcl w]
    puts $oh "# Generated from PrimeTime write_changes"
    set n 0
    while {[gets $fh line] >= 0} {
        if {[regexp {^size_cell\s+([^\s\[]+)} $line -> inst lib]} {
            set libcell [lindex [split $lib "/"] end]
            puts $oh "ecoChangeCell -inst $inst -cell $libcell"
            incr n
        } elseif {[regexp {^insert_buffer\s+([^\s\[]+)} $line -> pin
lib]} {
            set libcell [lindex [split $lib "/"] end]
            puts $oh "ecoAddRepeater -term $pin -cell $libcell"
            incr n
        }
    }
    close $fh
    close $oh
    puts "exported $n ECO commands to $dir/ (icc2, pt, innovus)"

    # Post-ECO summary for the sign-off note.
    redirect -file $dir/post_eco_qor.rpt { report_qor }
    redirect -file $dir/post_eco_global.rpt { report_global_timing }
}
```

`write_changes -format icctcl` produces commands IC Compiler II reads directly. For Innovus the script translates the `ptsh` format line by line: `size_cell` becomes `ecoChangeCell`, `insert_buffer` becomes `ecoAddRepeater`. Check the generated file against the Innovus version in use; option names have changed across releases.

The post-ECO `report_qor` and `report_global_timing` are the numbers to put in the ECO sign-off note next to the pre-ECO numbers from the same procs.

8.5 Using `fix_eco_timing` with before/after metrics

When the built-in fixer is acceptable, the value of a wrapper is the measurement around it: WNS, TNS and violator count before and after, printed together.

```
#-----
# File      : 08_fix_eco_wrapper.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Wrapper around fix_eco_timing / fix_eco_drc with before/after
# Tool      : PrimeTime (pt_shell), requires PrimeTime ECO license
# Usage     : source 08_fix_eco_wrapper.tcl ; run_fix_eco setup
#-----
```

```

proc wns_tns {{delay_type max}} {
    set paths [get_timing_paths -delay_type $delay_type -nworst 1 -max_paths 100000 -
slack_lesser_than 0]
    set wns 0.0
    set tns 0.0
    set n 0
    foreach_in_collection p $paths {
        set s [get_attribute $p slack]
        incr n
        set tns [expr {$tns + $s}]
        if {$s < $wns} { set wns $s }
    }
    return [list $wns $tns $n]
}

# type: setup | hold | drc
proc run_fix_eco {type args} {
    array set opt {-methods {size_cell} -buffer_list {} -slack_lesser_than 0 -cell_pattern {}}
    array set opt $args
    switch -- $type {
        setup { lassign [wns_tns max] w0 t0 n0 }
        hold { lassign [wns_tns min] w0 t0 n0 }
        drc { set w0 0; set t0 0; set n0 0 }
    }
    switch -- $type {
        setup {
            fix_eco_timing -type setup -methods $opt(-methods) \
                -slack_lesser_than $opt(-slack_lesser_than)
        }
        hold {
            set cmd [list fix_eco_timing -type hold -methods $opt(-methods)]
            if {$opt(-buffer_list) ne ""} { lappend cmd -buffer_list $opt(-buffer_list) }
            eval $cmd
        }
        drc {
            set cmd [list fix_eco_drc -type max_transition -methods $opt(-methods)]
            if {$opt(-buffer_list) ne ""} { lappend cmd -buffer_list $opt(-buffer_list) }
            eval $cmd
            set cmd [list fix_eco_drc -type max_capacitance -methods $opt(-methods)]
            if {$opt(-buffer_list) ne ""} { lappend cmd -buffer_list $opt(-buffer_list) }
            eval $cmd
        }
    }
    update_timing
    switch -- $type {
        setup { lassign [wns_tns max] w1 t1 n1 }
        hold { lassign [wns_tns min] w1 t1 n1 }
        drc { set w1 0; set t1 0; set n1 0 }
    }
    puts [format "%-6s before: WNS %.3f TNS %.3f N %d" $type $w0 $t0 $n0]
    puts [format "%-6s after : WNS %.3f TNS %.3f N %d" $type $w1 $t1 $n1]
}

```

Run `drc` first, then `setup`, then `hold`. Fixing DRV first often improves setup on its own; fixing hold last avoids re-breaking hold with setup upsizing. After the three passes, run `export_eco`.

Chapter 9. Reporting and data export

PrimeTime reports are written for a human reading one path. Tracking a project over 50 runs needs machine-readable output: CSV for spreadsheets and diffs, JSON for dashboards. Pure Tcl is enough for both; no package needs to be installed in `pt_shell`.

9.1 CSV and JSON in pure Tcl

```
#-----
# File      : 09_csv_json.tcl
# Author    : youbumkim@gmail.com
# Purpose   : CSV and JSON writers in pure Tcl (no packages needed)
# Tool      : PrimeTime (pt_shell) or any Tcl 8.x
# Usage     : source 09_csv_json.tcl
#-----

# CSV: quote a field if it contains a comma, quote, or newline.
proc csv_field {v} {
    if {[regexp {[",\n]} $v]} {
        return "\"[string map {"\" \"\"} $v]\""
    }
    return $v
}

proc csv_row {fields} {
    set out {}
    foreach f $fields { lappend out [csv_field $f] }
    return [join $out ","]
}

# Writes header (list) and rows (list of lists) to a file.
proc write_csv {file header rows} {
    set fh [open $file w]
    puts $fh [csv_row $header]
    foreach r $rows { puts $fh [csv_row $r] }
    close $fh
}

# Reads a CSV back into a list of lists (simple: no embedded newlines).
proc read_csv {file} {
    set fh [open $file r]
    set rows {}
    while {[gets $fh line] >= 0} {
        if {$line eq ""} { continue }
        set fields {}
        set cur ""
        set inq 0
        for {set i 0} {$i < [string length $line]} {incr i} {
            set ch [string index $line $i]
            if {$inq} {
                if {$ch eq "\"" && [string index $line [expr {$i+1}]] eq "\""} {
                    append cur "\""; incr i
                } elseif {$ch eq "\""} {
                    set inq 0
                } else {
                    append cur $ch
                }
            } else {
                if {$ch eq "\""} { set inq 1 }
                elseif {$ch eq ","} { lappend fields $cur; set cur "" }
                else { append cur $ch }
            }
        }
        lappend fields $cur
        lappend rows $fields
    }
    close $fh
    return $rows
}

# JSON helpers. Values are emitted as numbers when they look numeric.
proc json_str {s} {
    set s [string map {"\n" "\\n" "\"" "\\\"" "\t" "\\t" "\r" "\\r"} $s]
    return "\"$s\""
}
```

```

proc json_val {v} {
    if {[string is double -strict $v] && ![regexp {^-?0\d} $v]} { return $v }
    if {$v eq "true" || $v eq "false"} { return $v }
    return [json_str $v]
}

# dict -> JSON object (one level; nested values must already be JSON)
proc json_obj {kv {raw_keys {}}} {
    set parts {}
    foreach {k v} $kv {
        if {[lsearch -exact $raw_keys $k] >= 0} {
            lappend parts "[json_str $k]: $v"
        } else {
            lappend parts "[json_str $k]: [json_val $v]"
        }
    }
    return "\{[join $parts ", "]\}"
}

proc json_arr {items {raw 0}} {
    set parts {}
    foreach i $items {
        if {$raw} { lappend parts $i } else { lappend parts [json_val $i] }
    }
    return "\[[join $parts ", "]\]"
}

# Rows (list of lists) + header -> JSON array of objects, written to file.
proc write_json_rows {file header rows} {
    set objs {}
    foreach r $rows {
        set kv {}
        foreach h $header v $r { lappend kv $h $v }
        lappend objs [json_obj $kv]
    }
    set fh [open $file w]
    puts $fh [json_arr $objs 1]
    close $fh
}

```

csv_field quotes only when needed, so files stay readable. read_csv handles quoted fields with embedded commas, which appear when a pin name contains a comma from a generate block.

json_val emits numbers unquoted so downstream tools get numeric types. The 0?* guard keeps values like 007 as strings, since a leading zero is not valid JSON for a number.

9.2 One JSON per run

```

#-----
# File      : 09_signoff_summary.tcl
# Author    : youbumkim@gmail.com
# Purpose   : One JSON file per run with everything a dashboard needs
# Tool      : PrimeTime (pt_shell)
# Usage     : source 09_csv_json.tcl ; source 09_signoff_summary.tcl
#            write_signoff_summary summary.json func ss0p72v125c
#-----

proc group_metrics {delay_type} {
    set out {}
    foreach_in_collection g [get_path_groups] {
        set gn [get_object_name $g]
        set paths [get_timing_paths -group $g -delay_type $delay_type -nworst 1 \
            -max_paths 100000 -slack_lesser_than 0]
        set wns 0.0; set tns 0.0; set n 0
        foreach_in_collection p $paths {
            set s [get_attribute $p slack]

```

```

        incr n
        set tns [expr {$tns + $s}]
        if {$s < $wns} { set wns $s }
    }
    lappend out [json_obj [list group $gn wns [format %.4f $wns] tns [format %.4f $tns] nvp
$n]]
    }
    return $out
}

proc write_signoff_summary {file mode corner} {
    set setup [group_metrics max]
    set hold [group_metrics min]
    redirect -variable drv { report_constraint -all_violators -max_transition -max_capacitance -
nosplit }
    set n_drv [regexp -all {\(VIOLATED\)} $drv]
    set kv [list {}
    design [get_object_name [current_design]] {}
    mode $mode {}
    corner $corner {}
    date [clock format [clock seconds] -format "%Y-%m-%dT%H:%M:%S"] {}
    tool [get_app_var sh_product_version] {}
    setup [json_arr $setup 1] {}
    hold [json_arr $hold 1] {}
    drv_violators $n_drv
    set fh [open $file w]
    puts $fh [json_obj $kv {setup hold}]
    close $fh
    puts "wrote $file"
}

```

The file holds design, mode, corner, tool version, timestamp, per-path-group WNS/TNS/violator counts for setup and hold, and the DRV count. Any dashboard, wiki bot or spreadsheet import can read it. A directory of these files, one per run, is a complete timing history of a project.

`get_path_groups` returns the groups the timer uses, including the default group and any created with `group_path`. Iterating over groups rather than clocks matches what `report_timing` does by default.

9.3 Diff two runs

```

#-----
# File      : 09_run_diff.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Compare endpoint slack between two runs (CSV from path_table)
# Tool      : any Tcl 8.x (runs outside pt_shell too)
# Usage     : tclsh 09_run_diff.tcl old.csv new.csv 0.010
#-----

# Input CSVs must have header: slack,...,endpoint (endpoint = last column
# of bucket_violators output, or any file with 'slack' and 'endpoint').
proc load_ep_slack {file} {
    set fh [open $file r]
    set hdr [split [gets $fh] ","]
    set is [lsearch -exact $hdr slack]
    set ie [lsearch -exact $hdr endpoint]
    if {$is < 0 || $ie < 0} { error "file: need slack and endpoint columns" }
    array set m {}
    while {[gets $fh line] >= 0} {
        set f [split $line ","]
        set ep [lindex $f $ie]
        set s [lindex $f $is]
        if {[info exists m($ep)] || $s < $m($ep)} { set m($ep) $s }
    }
}

```

```

close $fh
return [array get m]
}

proc run_diff {old new {thresh 0.010}} {
    array set a [load_ep_slack $old]
    array set b [load_ep_slack $new]
    set improved {}; set degraded {}; set new_ep {}; set gone {}
    foreach ep [array names b] {
        if {[info exists a($ep)]} { lappend new_ep [list $ep $b($ep)]; continue }
        set d [expr {$b($ep) - $a($ep)}]
        if {$d > $thresh} { lappend improved [list $ep $a($ep) $b($ep) $d] }
        if {$d < -1 * $thresh} { lappend degraded [list $ep $a($ep) $b($ep) $d] }
    }
    foreach ep [array names a] {
        if {[info exists b($ep)]} { lappend gone [list $ep $a($ep)] }
    }
    puts "endpoints old=[array size a] new=[array size b]"
    puts "improved=[llength $improved] degraded=[llength $degraded] new=[llength $new_ep]"
    puts "fixed/gone=[llength $gone]"
    puts "\nDEGRADED (worst first)"
    foreach r [lsort -real -index 3 $degraded] {
        lassign $r ep o n d
        puts [format " %9.3f -> %9.3f (%+.3f) %s" $o $n $d $ep]
    }
    puts "\nNEW VIOLATORS"
    foreach r [lsort -real -index 1 $new_ep] {
        lassign $r ep n
        puts [format " %9.3f %s" $n $ep]
    }
}

if {[info exists argv] && [llength $argv] >= 2} {
    run_diff [lindex $argv 0] [lindex $argv 1] [expr {[llength $argv] > 2 ? [lindex $argv 2] : 0.010}]
}

```

The input is any CSV with `slack` and `endpoint` columns, so the violator CSV from Chapter 7 works directly. Endpoints present only in the new file are new violators; endpoints present only in the old file were fixed (or moved above the slack threshold). The degraded list is what a reviewer wants first after an ECO or a netlist drop.

The last block makes the file runnable with `tclsh` outside PrimeTime, so a continuous integration job can produce the diff without a licence.

9.4 report_timing bundles

Every team argues about `report_timing` options until someone writes them down. These procs are the written-down version: one full-detail report for path review, one end-only report for tracking, one PBA report for pre-ECO confirmation.

```

#-----
# File      : 09_custom_report_timing.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Standard report_timing options bundled for signoff review
# Tool      : PrimeTime (pt_shell)
# Usage     : source 09_custom_report_timing.tcl ; rt_full CLK 20 setup.rpt
#-----

# Full-detail report used for path review: clock expanded, input pins,
# nets, transition, capacitance, derates, and 3 significant digits.
proc rt_full {group nworst_paths out {delay_type max}} {
    redirect -file $out {
        report_timing -group $group -delay_type $delay_type \
            -max_paths $nworst_paths -nworst 1 \

```

```

        -path_type full_clock_expanded \N
        -input_pins -nets -transition_time -capacitance \N
        -derate -crpr -significant_digits 3 -nosplit
    }
    puts "wrote $out"
}

# Short end-only list used for tracking: one line per endpoint.
proc rt_endpoints {group max_paths out {delay_type max}} {
    redirect -file $out {
        report_timing -group $group -delay_type $delay_type \N
        -max_paths $max_paths -nworst 1 -path_type end -nosplit
    }
    puts "wrote $out"
}

# Path-based analysis for the worst N paths before ECO decisions.
proc rt_pba {group max_paths out} {
    redirect -file $out {
        report_timing -group $group -delay_type max -max_paths $max_paths \N
        -pba_mode path -path_type full_clock_expanded \N
        -input_pins -nets -transition_time -capacitance -nosplit
    }
    puts "wrote $out"
}

# Bundle for all path groups at once.
proc rt_all_groups {dir {max_paths 50}} {
    file mkdir $dir
    foreach_in_collection g [get_path_groups] {
        set gn [get_object_name $g]
        set safe [string map {/ _ * all} $gn]
        rt_full $gn $max_paths $dir/setup_${safe}.rpt max
        rt_full $gn $max_paths $dir/hold_${safe}.rpt min
        rt_endpoints $gn 2000 $dir/setup_${safe}_end.rpt max
    }
}

```

-path_type full_clock_expanded shows the clock network stage by stage, which is needed to see where launch and capture latency diverge. -derate and -crpr add columns showing the derate factor and the CRPR credit, so a reviewer does not have to infer them.

rt_all_groups writes setup, hold and end-only reports for every group with a filename-safe group name. ****default**** becomes all_default_all, which is ugly but unambiguous.

Chapter 10. Multi-corner, multi-mode

Signoff is never one corner. There are two ways to run many: distributed multi-scenario analysis (DMSA), where one master pt_shell coordinates slave processes and merges results, or independent single-scenario runs joined afterwards by script. Both are covered; the second needs no extra licence features and is often what a small team actually does.

10.1 DMSA bring-up from a corner table

```

#-----
# File      : 10_dmsa_setup.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Distributed multi-scenario (DMSA) bring-up from a corner table
# Tool      : PrimeTime DMSA (pt_shell -multi_scenario)
# Usage     : pt_shell -multi_scenario -f 10_dmsa_setup.tcl

```

```

#-----
# Corner table: name lib_db spef derate_late derate_early
set corners {
  {ss_rcworst stdcell_ss0p72v125c.db top.rcworst_125c.spef.gz 1.05 0.95}
  {ss_cworst stdcell_ss0p72v125c.db top.cworst_125c.spef.gz 1.05 0.95}
  {ff_rcbest stdcell_ff0p88vm40c.db top.rcbest_m40c.spef.gz 1.00 0.92}
}
set modes {func scan}

set_app_var multi_scenario_working_directory ./dmsa_work
set_app_var multi_scenario_merged_error_log ./dmsa_work/merged_errors.log
set_host_options -num_processes 6 -max_cores 4

# Every scenario shares the netlist read; corner-specific data is small.
foreach mode $modes {
  foreach c $corners {
    lassign $c cname lib spef dl de
    set fh [open ./dmsa_work/${mode}_${cname}.tcl w]
    puts $fh "set_app_var link_path \"* ./lib/$lib\""
    puts $fh "read_verilog ./netlist/top.v.gz"
    puts $fh "current_design top"
    puts $fh "link_design"
    puts $fh "read_parasitics -format spef ./spef/$spef"
    puts $fh "source ./sdc/top_${mode}.sdc"
    puts $fh "set_timing_derate -cell_delay -net_delay -late $dl"
    puts $fh "set_timing_derate -cell_delay -net_delay -early $de"
    puts $fh "set_propagated_clock \[all_clocks\]"
    close $fh
    create_scenario -name ${mode}_${cname} \
      -specific_data ./dmsa_work/${mode}_${cname}.tcl
  }
}

current_session -all
remote_execute { update_timing -full }

# Merged reports across all scenarios: worst per endpoint.
report_timing -max_paths 100 -nworst 1 -path_type end > ./dmsa_work/merged_setup_end.rpt
report_timing -delay_type min -max_paths 100 -nworst 1 -path_type end >
./dmsa_work/merged_hold_end.rpt
report_constraint -all_violators > ./dmsa_work/merged_constraint.rpt

```

The corner table is the only thing to edit. The script writes one scenario file per mode/corner combination, creates the scenarios, and runs `update_timing` in every slave with one `remote_execute`. Reports run at the master level merge across scenarios: the merged end-only report shows each endpoint at its worst scenario.

`set_host_options -num_processes` is the number of slaves. Each slave holds a full copy of the design; size the count by memory, not by cores.

10.2 Collect per-scenario data

`remote_execute` sends a Tcl block to every slave in the current session. Slaves cannot return values directly, so the pattern is: each slave writes a small file named after `current_scenario`, and the master reads them all.

```

#-----
# File      : 10_scenario_loop.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Run a per-scenario proc remotely and collect results
# Tool      : PrimeTime DMSA (pt_shell -multi_scenario)
# Usage     : source 10_scenario_loop.tcl ; collect_wns_per_scenario
#-----

```

```

# Each remote slave writes one CSV line; the master joins them.
proc collect_wns_per_scenario {{dir ./dmsa_work/wns}} {
    file mkdir $dir
    current_session -all
    # Braces would keep $dir unsubstituted on the master; bake it in first.
    set body [string map [list @DIR@ $dir] {
        set paths [get_timing_paths -delay_type max -nworst 1 -max_paths 100000 -
slack_lesser_than 0]
        set wns 0.0; set tns 0.0; set n 0
        foreach_in_collection p $paths {
            set s [get_attribute $p slack]
            incr n; set tns [expr {$tns + $s}]
            if {$s < $wns} { set wns $s }
        }
        set fh [open @DIR@[current_scenario].csv w]
        puts $fh "[current_scenario],$wns,$tns,$n"
        close $fh
    }]
    remote_execute $body
    puts [format "%-24s %9s %10s %7s" SCENARIO WNS TNS NVP]
    foreach f [lsort [glob -nocomplain $dir/*.csv]] {
        set fh [open $f r]
        lassign [split [gets $fh] ","] sc w t n
        close $fh
        puts [format "%-24s %9.3f %10.3f %7d" $sc $w $t $n]
    }
}

# Which scenario is worst for a given endpoint? Useful before ECO.
proc worst_scenario_for {endpoint {dir ./dmsa_work/ep}} {
    file mkdir $dir
    current_session -all
    remote_execute "
        set p \[get_timing_paths -to $endpoint -nworst 1 -max_paths 1\]
        set fh \[open $dir/\[current_scenario\].txt w\]
        if {\$p ne \"\"} { puts \$fh \"\[current_scenario\] \[get_attribute \$p slack\]\" } else
{ puts \$fh \"\[current_scenario\] NA\" }
        close \$fh
    "
    set rows {}
    foreach f [glob -nocomplain $dir/*.txt] {
        set fh [open $f r]
        lassign [gets $fh] sc s
        close $fh
        if {[string is double -strict $s]} { lappend rows [list $sc $s] }
    }
    set rows [lsort -real -index 1 $rows]
    foreach r $rows { puts [format " %-24s %9.3f" [lindex $r 0] [lindex $r 1]] }
    return [lindex $rows 0]
}

```

`collect_wns_per_scenario` gives one line per scenario, which answers the first question of any review: is one corner much worse than the rest. `worst_scenario_for` answers the question that comes before any ECO: in which scenario must this endpoint be fixed, and which scenarios must be checked afterwards.

Both procs bake master-side values into the block before sending it (`string map` in the first, a double-quoted string in the second) because the block is evaluated in the slave, where the master's variables do not exist. A brace-quoted block containing `$dir` would fail in every slave with an undefined variable.

10.3 Merge independent runs

Without DMSA, each corner runs in its own `pt_shell` and writes its own violator CSV (recipe 7.1). `merge_runs` joins them into one worst-per-endpoint table with the corner that

produced the worst value and the number of corners in which the endpoint fails.

```
#-----
# File      : 10_merge_csv_runs.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Merge per-corner violator CSVs into one worst-per-endpoint table
# Tool      : any Tcl 8.x (runs outside pt_shell)
# Usage     : tclsh 10_merge_csv_runs.tcl merged.csv rpt/*/violators.csv
#-----

# Input files come from bucket_violators -out. The corner name is taken
# from the parent directory of each CSV.
proc merge_runs {out files} {
    array set worst {}
    array set corner {}
    array set ncorner {}
    foreach f $files {
        set cname [file tail [file dirname $f]]
        set fh [open $f r]
        set hdr [split [gets $fh] ","]
        set is [lsearch -exact $hdr slack]
        set ie [lsearch -exact $hdr endpoint]
        while {[gets $fh line] >= 0} {
            set fl [split $line ","]
            set ep [lindex $fl $ie]
            set s [lindex $fl $is]
            if {[info exists ncorner($ep)]} { set ncorner($ep) 0 }
            incr ncorner($ep)
            if {[info exists worst($ep)] || $s < $worst($ep)} {
                set worst($ep) $s
                set corner($ep) $cname
            }
        }
        close $fh
    }
    set rows {}
    foreach ep [array names worst] {
        lappend rows [list $worst($ep) $corner($ep) $ncorner($ep) $ep]
    }
    set rows [lsort -real -index 0 $rows]
    set oh [open $out w]
    puts $oh "worst_slack,worst_corner,n_corners_failing,endpoint"
    foreach r $rows { puts $oh [join $r ","] }
    close $oh
    puts "merged [llength $rows] endpoints from [llength $files] files -> $out"
}

if {[info exists argv] && [llength $argv] >= 2} {
    merge_runs [lindex $argv 0] [lrange $argv 1 end]
}
```

The corner name is taken from the directory containing each CSV, so a layout of `rpt/<mode>_<corner>/violators.csv` works without any extra argument. The script runs under `tclsh`.

An endpoint failing in one corner only is usually a corner-specific issue (a hold violation in the fast corner, a setup violation with `cworst` parasitics). An endpoint failing in every corner is a logic depth problem.

Chapter 11. Performance and hygiene

A script that takes an hour gets run once a week. A script that takes a minute gets run after every change. This chapter is about the second kind.

11.1 Measure first

```
#-----
# File      : ll_timing_util.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Measure runtime and memory of any command block
# Tool      : PrimeTime (pt_shell)
# Usage     : source ll_timing_util.tcl ; timeit "worst paths" { get_timing_paths -max_paths 10000 }
#-----

proc mem_mb {} {
    # sh_memory_usage is not available in every release; fall back to /proc
    if {![catch {set m [get_app_var sh_memory_usage]}] && [string is double -strict $m]} {
        return [expr {$m / 1024.0}]
    }
    if {[file exists /proc/self/status]} {
        set fh [open /proc/self/status r]
        while {[gets $fh line] >= 0} {
            if {[regexp {\^VmRSS:\s+(\d+)} $line -> kb]} { close $fh; return [expr {$kb / 1024.0}] }
        }
        close $fh
    }
    return 0
}

proc timeit {label body} {
    set t0 [clock milliseconds]
    set m0 [mem_mb]
    set rc [catch {uplevel 1 $body} result]
    set dt [expr {[clock milliseconds] - $t0} / 1000.0]
    set m1 [mem_mb]
    puts [format "TIMEIT %-30s %8.2fs  mem %8.0f -> %8.0f MB" $label $dt $m0 $m1]
    if {$rc} { return -code error $result }
    return $result
}
```

`timeit` prints wall-clock time and resident memory before and after a block. Wrap the slow-looking part of any new script in it before optimising; the slow part is usually not the one you expect.

11.2 Patterns that are slow, and their fast equivalents

```
#-----
# File      : ll_fast_patterns.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Slow versus fast versions of common queries
# Tool      : PrimeTime (pt_shell)
# Usage     : source ll_fast_patterns.tcl
#-----

# SLOW: one get_timing_paths call per endpoint (N timing queries).
proc worst_slack_per_endpoint_slow {endpoints} {
    set out {}
    foreach_in_collection e $endpoints {
        set p [get_timing_paths -to $e -nworst 1 -max_paths 1]
        if {$p ne ""} { lappend out [list [get_object_name $e] [get_attribute $p slack]] }
    }
    return $out
}
```

```

}

# FAST: one query, filter afterwards (one timing query).
proc worst_slack_per_endpoint_fast {endpoints} {
    set out {}
    set paths [get_timing_paths -to $endpoints -nworst 1 -max_paths [sizeof_collection
$endpoints] -slack_lesser_than 1e9]
    foreach_in_collection p $paths {
        lappend out [list [get_object_name [get_attribute $p endpoint]] [get_attribute $p slack]]
    }
    return $out
}

# FASTEST when timing_save_pin_arrival_and_slack is on: no path query.
proc worst_slack_per_endpoint_attr {endpoints} {
    set out {}
    foreach_in_collection e $endpoints {
        lappend out [list [get_object_name $e] [get_attribute -quiet $e max_slack]]
    }
    return $out
}

# SLOW: get_cells inside a loop re-parses the pattern every iteration.
proc mark_dont_touch_slow {names} {
    foreach n $names { set_dont_touch [get_cells $n] }
}

# FAST: build one collection, apply once.
proc mark_dont_touch_fast {names} {
    set_dont_touch [get_cells $names]
}

# SLOW: string concatenation of a large report in a variable.
# FAST: redirect straight to a file.
proc big_report_fast {out} {
    redirect -file $out { report_timing -max_paths 100000 -nworst 1 -path_type end -nosplit }
}

# Filter in the query, not in Tcl: -filter runs in C, foreach runs in Tcl.
proc seq_cells_fast {} {
    return [get_cells -hierarchical -filter "is_sequential == true && is_hierarchical == false"]
}

```

The four rules behind these examples:

1. **One query, not N.** `get_timing_paths -to $collection` with a large `-max_paths` is one timer call. The same in a loop is N timer calls, each with its own setup cost.
2. **Filter in the query.** `-filter` runs inside the tool. A Tcl loop with `get_attribute` per object runs in the interpreter with a command dispatch per attribute.
3. **Attributes over paths when possible.** With `timing_save_pin_arrival_and_slack`, `max_slack` on a pin is a memory read. A path query is a graph traversal.
4. **Files over variables for large output.** `redirect -variable` builds the whole string in memory and then copies it again on assignment.

Two more that have no code example:

- `update_timing` is incremental after netlist edits. Do not call `update_timing -full` inside an ECO loop.
- `get_cells -hierarchical` with no filter on a full chip returns millions of objects. Always attach a `-filter` or a name pattern.

11.3 Log like a batch job

```
#-----
# File      : ll_logging.tcl
# Author    : youbumkim@gmail.com
# Purpose   : Structured run log with levels, written to file and stdout
# Tool      : PrimeTime (pt_shell) or any Tcl 8.x
# Usage     : source ll_logging.tcl ; log_open run.log ; log INFO "started"
#-----

set ::log_fh ""
set ::log_level 1      ;# 0=DEBUG 1=INFO 2=WARN 3=ERROR

proc log_open {file} {
    set ::log_fh [open $file a]
    log INFO "log opened: $file"
}

proc log_close {} {
    if {$::log_fh ne ""} { close $::log_fh; set ::log_fh "" }
}

proc log {level msg} {
    array set lv {DEBUG 0 INFO 1 WARN 2 ERROR 3}
    if {$lv($level) < $::log_level} { return }
    set line [format "%s %-5s %s" [clock format [clock seconds] -format "%H:%M:%S"] $level $msg]
    puts $line
    if {$::log_fh ne ""} { puts $::log_fh $line; flush $::log_fh }
}

# Run a body and log its start, end, duration and any error.
proc log_step {name body} {
    log INFO "start $name"
    set t0 [clock seconds]
    if {[catch {uplevel 1 $body} err]} {
        log ERROR "$name failed: $err"
        return -code error $err
    }
    log INFO "done $name ([expr {[clock seconds] - $t0}]s)"
}
```

Levels make a long log greppable: `grep ERROR run.log` is the whole post-mortem. `log_step` records the duration of each step so that when the batch takes twice as long as last week, the log says which step grew.

Flush after every line. A batch that is killed by the scheduler loses everything buffered since the last flush, which is exactly the part you need.

11.4 Script hygiene checklist

- Header with file name, author, purpose, tool, usage. Every script in this book has one; copy the format.
- No hard-coded paths inside procs. Paths go in a configuration block or in options.
- `-quiet` on every `get_*` and `get_attribute` where an empty result is legitimate. Without it, an empty result is an error message in the log and, with `sh_continue_on_error false`, a stopped batch.
- Return values, not just printed text. Every table proc here returns its rows.
- One proc per job. A proc that reports and fixes cannot be used to report only.

Appendix A. Command and attribute quick reference

A.1 Query commands

Command	Returns	Notes
<code>get_cells [-hierarchical] [-filter expr] pattern</code>	cells	<code>-hierarchical</code> searches all levels
<code>get_pins [-of_objects cells] [-leaf]</code>	pins	<code>-leaf</code> skips hierarchical pins
<code>get_nets [-of_objects pins]</code>	nets	
<code>get_ports, all_inputs, all_outputs</code>	ports	
<code>get_clocks, all_clocks</code>	clocks	
<code>all_registers [-clock_pins -data_pins] [-clock c]</code>	pins	most reliable register selection
<code>all_fanin -to p -flat [-startpoints_only]</code>	pins	
<code>all_fanout -from p -flat [-endpoints_only]</code>	pins	
<code>get_timing_paths [options]</code>	timing_path	see Chapter 4
<code>get_path_groups</code>	path groups	
<code>get_lib_cells lib/cell</code>	lib_cell	
<code>get_alternative_lib_cells cell</code>	lib_cell	for ECO sizing

A.2 Collection commands

Command	Purpose
<code>foreach_in_collection var coll body</code>	iterate
<code>sizeof_collection coll</code>	count
<code>index_collection coll i</code>	i-th element
<code>add_to_collection a b [-unique]</code>	union
<code>remove_from_collection a b</code>	difference
<code>compare_collections -intersect a b</code>	intersection
<code>filter_collection coll expr</code>	filter after query
<code>sort_collection [-descending] coll attr</code>	sort
<code>get_object_name coll</code>	names as list

A.3 Timing path attributes used in this book

Attribute	Type	Meaning
slack	float or INFINITY	required minus arrival
startpoint, endpoint	object	pin or port
startpoint_clock, endpoint_clock	clock	may be empty
path_group	path group	
arrival, required	float	at the endpoint
points	collection of timing_point	ordered along the path
startpoint_clock_latency, endpoint_clock_latency	float	as used for this path
common_path_pessimism	float	CRPR credit
path_type	string	max or min

A.4 Application variables that change results

Variable	Recommended	Effect
timing_remove_clock_reconvergence_pessimism	true	CRPR
timing_save_pin_arrival_and_slack	true	pin slack attributes
timing_report_unconstrained_paths	true	show unconstrained in reports
timing_enable_through_paths	true	- through in exceptions
si_enable_analysis	per flow	crosstalk (PrimeTime SI)
sh_continue_on_error	true in batch	do not abort sourced files

A.5 ECO commands

Command	Purpose
size_cell cell lib_cell	change lib cell
insert_buffer pin lib_cell	add buffer at pin
remove_buffer cell	remove buffer
write_changes -format icctl ptsh dctl -output f	export
fix_eco_timing -type setup hold -methods {size_cell insert_buffer}	automatic
fix_eco_drc -type max_transition max_capacitance	automatic DRV

Appendix B. Equivalents in Cadence Tempus

The scripts target PrimeTime. The same recipes map to Tempus with these substitutions; the logic of each proc is unchanged.

PrimeTime	Tempus
get_timing_paths	report_timing -collection
get_attribute obj attr	get_property obj attr
foreach_in_collection	foreach over the returned list
sizeof_collection	llength
-slack_less_than	-max_slack
points on a path	timing_points
startpoint_clock	launching_clock
endpoint_clock	capturing_clock
all_registers -clock_pins	all_registers -clock_pins
set_app_var	set_global / setAnalysisMode
write_changes	ecoChangeCell, ecoAddRepeater (direct)

Tempus returns Tcl lists rather than collections, so the list-versus-collection pitfalls of Chapter 1 do not apply, but a path object is still a handle whose attributes are read with `get_property`.

Appendix C. Script file list

All scripts are in the `scripts/` directory of this package. Each file is self-contained except where noted.

File	Depends on
01_collection_basics.tcl	none
01_attr_helpers.tcl	none
01_safe_source.tcl	none
01_args_parser.tcl	none
02_session_setup.tcl	none (edit config block)
02_env_report.tcl	none
02_check_timing_parse.tcl	none
02_sdc_echo_filter.tcl	none
03_find_cells.tcl	none
03_fanin_fanout.tcl	none
03_list_attrs.tcl	none
04_path_table.tcl	none
04_slack_histogram.tcl	none

File	Depends on
04_worst_per_clock_pair.tcl	none
04_path_classify.tcl	none
04_path_points.tcl	none
05_clock_summary.tcl	none
05_latency_skew.tcl	none
05_generated_clock_check.tcl	none
06_unconstrained_endpoints.tcl	timing_save_pin_arrival_and_slack
06_exception_audit.tcl	none
06_io_constraint_coverage.tcl	timing_save_pin_arrival_and_slack
06_case_and_disable.tcl	none
07_bucket_violators.tcl	none
07_common_cells.tcl	none
07_drv_summary.tcl	none
08_eco_candidates.tcl	ECO capability
08_eco_upsize.tcl	ECO capability
08_hold_buffer.tcl	ECO capability, timing_save_pin_arrival_and_slack
08_write_eco.tcl	ECO capability
08_fix_eco_wrapper.tcl	ECO capability
09_csv_json.tcl	none
09_signoff_summary.tcl	09_csv_json.tcl
09_run_diff.tcl	none (tclsh)
09_custom_report_timing.tcl	none
10_dmsa_setup.tcl	DMSA (edit corner table)
10_scenario_loop.tcl	DMSA
10_merge_csv_runs.tcl	none (tclsh)
11_timing_util.tcl	none
11_fast_patterns.tcl	none
11_logging.tcl	none